

LAPORAN TUGAS BESAR 2

IF2211 Strategi Algoritma

Pemanfaatan Algoritma BFS dan DFS dalam Mekanisme

Penelusuran CSS pada pohon Document Object Model

Semester 2 Tahun 2025/2026



Disusun oleh:

Tuak

Juan Oloando Simanungkalit 13524032

Ariel Cornelius Sitorus 13524085

Manuel Thimoty Silalahi 13524102

Laboratorium Ilmu dan Rekayasa Komputasi

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Daftar Isi

BAB 1	
DESKRIPSI TUGAS.....	3
BAB 2	
TEORI DASAR.....	5
2.1. Pohon dan Graf.....	5
2.2. Document Object Model (DOM).....	5
2.2.1. Struktur Pohon DOM.....	6
2.3. Selektor CSS.....	6
2.3.1. Jenis Selektor CSS yang Didukung.....	7
2.4. Algoritma Breadth-First Search (BFS).....	7
2.4.1. Cara kerja BFS.....	8
2.4.2. Kompleksitas BFS.....	8
2.5. Algoritma Depth-First Search (DFS).....	9
2.5.1. Cara kerja DFS.....	9
2.5.2. Kompleksitas DFS.....	9
2.6. Algoritma LCA dengan Binary Lifting.....	10
2.6.1. Binary Lifting.....	10
2.6.2. LCA dengan Binary Lifting.....	11
2.7. Multithreading.....	11
Bab 3	
Aplikasi BFS dan DFS Pada DOM.....	12
3.1. Pendekatan Umum.....	12
3.2. Proses Parsing HTML ke Pohon DOM.....	12
3.2.1. Tokenisasi (Lexing).....	12
3.2.2. Pembangunan Pohon (Tree Building).....	13
3.3. Pencocokan Selektor CSS (matchesSelector).....	13
3.3.1. Selektor Sederhana (matchesSimpleSelector).....	13
3.3.2. Selektor Kompleks dengan Combinator.....	13
3.4. Aplikasi BFS pada DOM.....	14
3.5. Aplikasi DFS pada DOM.....	14
3.6. Perbandingan Hasil Traversal.....	14
Bab 4	
Implementasi dan Pengujian.....	16
Bab 5	
Kesimpulan dan Saran.....	17
5.1. Kesimpulan.....	17
5.2. Saran.....	17
Daftar Pustaka.....	18
LAMPIRAN.....	19

BAB 1

DESKRIPSI TUGAS

Tugas Besar IF2211 Strategi Algoritma Semester II Tahun 2025/2026 ini bertujuan untuk mengimplementasikan algoritma penelusuran *tree*, yaitu Breadth First Search (BFS) dan Depth First Search (DFS), dalam proses pencarian elemen pada struktur Document Object Model (DOM) berdasarkan CSS selector. DOM merupakan representasi terstruktur dari dokumen HTML dalam bentuk pohon (*tree*) yang memungkinkan setiap elemen pada halaman web untuk diakses.

Dalam struktur DOM, setiap elemen HTML direpresentasikan sebagai node dalam sebuah pohon dengan hubungan parent, child, dan sibling. Umumnya, struktur HTML mempunyai elemen akar `<html>` yang terdiri dari dua bagian utama, yaitu `<head>` dan `<body>`. Bagian `<head>` berisi metadata, sedangkan `<body>` berisi konten utama yang ditampilkan kepada pengguna.

Untuk menentukan elemen yang ingin diproses atau ditampilkan, digunakan CSS Selector. CSS Selector ini merupakan mekanisme untuk memilih elemen tertentu dalam struktur DOM berdasarkan kriteria tertentu seperti tag, class, dan id. Oleh karena itu, pencarian elemen berdasarkan CSS selector dapat dimodelkan sebagai suatu permasalahan penelusuran pada struktur pohon.

Pada tugas besar ini, mahasiswa diminta untuk membuat aplikasi traversal pohon HTML (DOM Tree) yang menggunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS) untuk melakukan pencarian elemen berdasarkan CSS selector pada struktur DOM. Aplikasi yang dibuat harus memenuhi spesifikasi berikut:

- Membuat aplikasi berbasis web yang terdiri dari frontend bebas dan backend dalam bahasa Go, C#, Rust, atau Typescript yang mengimplementasikan algoritma BFS dan DFS untuk melakukan penelusuran CSS pada pohon DOM
- Aplikasi dapat melakukan scraping HTML dari sebuah website melalui URL yang diberikan oleh pengguna
- Menerima masukan berupa URL website atau teks HTML, pilihan algoritma (DFS/BFS), CSS Selector, dan jumlah hasil yang diinginkan (Top n kemunculan/semua kemunculan)

- Melakukan parsing HTML menjadi struktur DOM tree
- Menampilkan visualisasi pohon DOM serta informasi kedalamannya
- Menampilkan jalur traversal algoritma
- Menampilkan waktu pencarian dan jumlah node yang dikunjungi
- Menyimpan traversal log dari proses penelusuran

BAB 2

TEORI DASAR

2.1. Pohon dan Graf

Dalam ilmu komputer, graf adalah struktur data yang terdiri dari himpunan simpul (vertices/nodes) dan himpunan sisi (edges) yang menghubungkan pasangan simpul. Pohon (tree) adalah kasus khusus graf tak berarah dan terhubung yang tidak memiliki siklus. Pohon dengan n simpul memiliki tepat $n-1$ sisi.

Setiap pohon berakar (rooted tree) memiliki satu simpul khusus yang disebut akar (root). Simpul-simpul lain memiliki hubungan hierarkis: parent (orang tua) dan children (anak). Simpul yang tidak memiliki anak disebut leaf (daun). Kedalaman (depth) suatu simpul adalah panjang jalur dari akar ke simpul tersebut.

2.2.Document Object Model (DOM)

Document Object Model (DOM) adalah representasi terstruktur dari dokumen HTML yang memodelkan setiap elemen dalam bentuk pohon (tree). Struktur ini memungkinkan program untuk mengakses, memodifikasi, dan memanipulasi konten, struktur, maupun gaya visual halaman web secara dinamis. Tanpa adanya DOM, sebuah website akan bersifat statis dan tidak dapat merespons perubahan setelah dimuat oleh web browser. DOM memiliki peran sebagai berikut:

- **Manipulasi Konten secara Dinamis:** DOM memfasilitasi perubahan teks, gambar, atau elemen HTML lainnya secara *real-time* tanpa mengharuskan pengguna melakukan pemuatan ulang (*reload*) seluruh halaman.
- **Modifikasi Struktur Dokumen:** Melalui struktur pohon yang hierarkis, elemen HTML dapat ditambah, dihapus, atau diganti secara fleksibel sesuai dengan kebutuhan logika aplikasi.
- **Kontrol Visual dan Gaya (CSS):** DOM memungkinkan program untuk mengakses serta mengubah properti gaya visual (CSS) pada elemen tertentu, sehingga tampilan antarmuka dapat beradaptasi secara dinamis terhadap aksi pengguna.
- **Manajemen Interaksi Pengguna (*Event Handling*):** DOM berfungsi sebagai mekanisme untuk mendeteksi dan merespons berbagai aktivitas pengguna, seperti input data, penekanan tombol, atau pergerakan kursor.

- Optimalisasi Performa Aplikasi Web: Penggunaan DOM yang efektif mempercepat proses pengelolaan data pada aplikasi berbasis *Single Page Application* (SPA), sehingga pengembangan aplikasi menjadi lebih efisien dan responsif.

2.2.1. Struktur Pohon DOM

Dalam pohon DOM, setiap tag HTML menjadi sebuah simpul elemen. Tag-tag yang bersarang menjadi anak dari tag yang melingkupinya. Contoh struktur HTML sederhana:

```
<html>
  <head><title>Contoh</title></head>
  <body>
    <div id="konten">
      <p class="teks">Hello World</p>
    </div>
  </body>
</html>
```

Struktur di atas menghasilkan pohon DOM: html adalah akar, dengan dua anak: head dan body. body memiliki satu anak div, yang memiliki satu anak p.

2.3. Selektor CSS

CSS Selector adalah mekanisme dalam bahasa CSS yang digunakan untuk menentukan elemen HTML mana yang akan diberi gaya (style) tertentu pada sebuah halaman web. Selector bekerja dengan cara “memilih” elemen berdasarkan nama tag, atribut, class, id, atau hubungan antar elemen dalam struktur DOM (Document Object Model).

Dengan menggunakan CSS Selector, pengembang dapat mengatur tampilan halaman secara spesifik dan efisien, misalnya hanya menargetkan elemen tertentu tanpa mempengaruhi elemen lain. Contohnya, selector p akan memilih semua elemen paragraf, .container akan memilih semua elemen dengan class tersebut, dan #header akan memilih elemen dengan id tertentu. Selain itu, terdapat juga selector yang lebih kompleks seperti descendant selector, child selector, dan pseudo-class yang memungkinkan pengaturan gaya berdasarkan kondisi atau posisi elemen. Dengan demikian, CSS Selector merupakan

komponen penting dalam pemisahan antara struktur (HTML) dan tampilan (CSS) dalam pengembangan web.

2.3.1. Jenis Selektor CSS yang Didukung

Jenis Selektor	Contoh & Keterangan
Tag Selector	div — Mencocokkan semua elemen <div>
Class Selector	.container — Mencocokkan elemen dengan class 'container'
ID Selector	#header — Mencocokkan elemen dengan id 'header'
Descendant Combinator	div p — Mencocokkan <p> di dalam <div> mana pun
Child Combinator (>)	div > p — Mencocokkan <p> yang merupakan anak langsung <div>
Adjacent Sibling (+)	h1 + p — Mencocokkan <p> yang langsung mengikuti <h1>
General Sibling (~)	h1 ~ p — Mencocokkan semua <p> yang merupakan saudara setelah <h1>

2.4. Algoritma Breadth-First Search (BFS)

Breadth-First Search (BFS) merupakan algoritma yang digunakan untuk menelusuri graf atau pohon dengan cara melebar, yaitu mengunjungi semua simpul pada

satu tingkat terlebih dahulu sebelum berpindah ke tingkat berikutnya. Dalam implementasinya, BFS menggunakan struktur data antrian (queue) untuk mengatur urutan simpul yang akan diproses. Algoritma ini sangat efektif untuk mencari jalur terpendek pada graf yang tidak berbobot, karena simpul pertama yang ditemukan dijamin memiliki jarak minimum dari simpul awal.

2.4.1. Cara kerja BFS

BFS menggunakan struktur data antrian (queue) yang bersifat FIFO (First In, First Out). Langkah-langkah algoritmanya adalah sebagai berikut:

1. Masukkan simpul akar ke dalam antrian dan tandai sebagai dikunjungi.
2. Selama antrian tidak kosong: ambil (dequeue) simpul paling depan.
3. Kunjungi simpul tersebut dan proses (dalam konteks ini: cek apakah cocok dengan selektor CSS).
4. Masukkan semua anak (children) simpul tersebut ke dalam antrian.
5. Ulangi langkah 2-4 hingga antrian kosong atau target ditemukan.

Properti penting BFS: simpul-simpul dikunjungi dalam urutan jarak meningkat dari akar. Semua simpul di kedalaman d dikunjungi sebelum simpul di kedalaman $d+1$.

2.4.2. Kompleksitas BFS

Aspek	Nilai
Kompleksitas Waktu	$O(V + E)$, V = jumlah node, E = jumlah sisi
Kompleksitas Ruang	$O(V)$ untuk antrian dan array visited
Pada Pohon DOM	$O(N)$ karena pohon memiliki $N-1$ sisi untuk N node

Keunggulan	Menemukan elemen pada kedalaman paling dangkal terlebih dahulu
Kelemahan	Membutuhkan memori lebih besar untuk pohon lebar

2.5. Algoritma Depth-First Search (DFS)

Depth-First Search (DFS) merupakan algoritma penelusuran graf atau pohon yang bekerja secara mendalam, yaitu dengan mengikuti satu cabang hingga mencapai simpul terdalam sebelum kembali (backtracking) dan melanjutkan ke cabang lainnya. DFS biasanya diimplementasikan menggunakan struktur data stack atau dengan pendekatan rekursi. Algoritma ini sering digunakan untuk berbagai keperluan seperti mendeteksi siklus, melakukan topological sorting, serta menjelajahi seluruh kemungkinan jalur dalam suatu graf.

2.5.1. Cara kerja DFS

DFS menggunakan struktur data tumpukan (stack) yang bersifat LIFO (Last In, First Out). Langkah-langkah algoritmanya:

1. Masukkan simpul akar ke dalam tumpukan.
2. Selama tumpukan tidak kosong: ambil (pop) simpul paling atas.
3. Kunjungi simpul tersebut dan proses.
4. Masukkan semua anak simpul tersebut ke tumpukan (dalam urutan terbalik agar anak pertama diproses duluan).
5. Ulangi langkah 2-4 hingga tumpukan kosong atau target ditemukan.

2.5.2. Kompleksitas DFS

Aspek	Nilai
Kompleksitas Waktu	$O(V + E)$, sama dengan BFS

Kompleksitas Ruang	$O(h)$, h = kedalaman pohon (height)
Pada Pohon DOM	$O(N)$ karena pohon memiliki $N-1$ sisi
Keunggulan	Hemat memori untuk pohon sempit dan dalam
Kelemahan	Tidak menjamin menemukan elemen paling dangkal

2.6. Algoritma LCA dengan Binary Lifting

Lowest Common Ancestor (LCA) adalah konsep dalam struktur data pohon yang digunakan untuk mencari nenek moyang bersama terdekat dari dua simpul. Dengan kata lain, LCA dari dua node adalah node terdalam (paling dekat dengan kedua node tersebut) yang menjadi leluhur bagi keduanya. Masalah LCA sering muncul dalam berbagai aplikasi seperti analisis struktur hierarki, pemrosesan query pada pohon, dan algoritma graf. Tanpa optimasi, pencarian LCA dapat dilakukan dengan menelusuri parent dari masing-masing node hingga ditemukan titik pertemuan, namun cara ini kurang efisien untuk banyak query.

2.6.1. Binary Lifting

Binary Lifting adalah teknik optimasi yang digunakan pada pohon untuk mempercepat proses pencarian ancestor (leluhur) dari suatu node. Ide utamanya adalah melakukan preprocessing dengan menyimpan informasi leluhur pada jarak tertentu, yaitu dalam bentuk pangkat dua (2^1 , 2^2 , 2^3 , dan seterusnya). Dengan cara ini, kita dapat “melompat” ke atas pohon dalam langkah yang lebih besar, bukan satu per satu. Struktur ini biasanya disimpan dalam tabel dua dimensi, di mana setiap entri menyimpan leluhur ke- 2^k dari suatu node. Proses preprocessing ini memerlukan waktu $O(N \log N)$, sedangkan query untuk mencari ancestor dapat dilakukan dalam waktu $O(\log N)$.

2.6.2. LCA dengan Binary Lifting

LCA dengan Binary Lifting merupakan metode yang menggabungkan konsep LCA dan teknik Binary Lifting untuk mendapatkan solusi yang efisien. Langkah pertama adalah menyamakan kedalaman kedua node dengan cara mengangkat node yang lebih dalam menggunakan Binary Lifting. Setelah berada pada kedalaman yang sama, kedua node kemudian diangkat secara bersamaan dari tingkat tertinggi ke bawah hingga ditemukan titik di mana leluhur mereka berbeda. Node terakhir sebelum perbedaan tersebut merupakan LCA dari kedua node. Dengan pendekatan ini, setiap query LCA dapat dijawab dalam waktu $O(\log N)$, sehingga sangat efisien untuk kasus dengan banyak query.

2.7. Multithreading

Multithreading adalah teknik dalam pemrograman yang memungkinkan sebuah program menjalankan beberapa proses atau tugas secara bersamaan dengan memanfaatkan lebih dari satu thread dalam satu waktu. Setiap thread merupakan unit eksekusi yang dapat berjalan secara paralel, sehingga multithreading sangat berguna untuk meningkatkan performa, terutama pada operasi yang berat atau membutuhkan banyak komputasi. Dengan adanya multithreading, suatu aplikasi dapat tetap responsif karena pekerjaan dapat dibagi ke beberapa thread tanpa harus menunggu satu proses selesai terlebih dahulu.

Bab 3

Aplikasi BFS dan DFS Pada DOM

3.1. Pendekatan Umum

Penelusuran CSS pada pohon DOM secara konseptual identik dengan penelusuran graf berakar. Setiap tag HTML adalah simpul, dan hubungan parent-child membentuk sisi-sisi pohon. Permasalahan penelusurannya adalah: "temukan semua simpul yang atributnya (tagName, className, idName) cocok dengan pola selektor CSS yang diberikan." Permasalahan ini ekuivalen dengan mencari suatu node pada sebuah graf/tree.

Standar browser sendiri menggunakan DFS (depth-first pre-order traversal) untuk metode `querySelector()` dan `querySelectorAll()`, sebagaimana disebutkan dalam dokumentasi MDN: "The matching is done using depth-first pre-order traversal of the document's nodes" (MDN, tahun 2024). Aplikasi ini mengimplementasikan keduanya (BFS dan DFS) sehingga pengguna dapat membandingkan perilaku dan urutan hasil dari kedua algoritma.

3.2. Proses Parsing HTML ke Pohon DOM

Sebelum BFS/DFS dapat dijalankan, HTML mentah harus diubah menjadi representasi pohon. Proses ini dilakukan dalam dua tahap di file [parser.ts](#):

3.2.1. Tokenisasi (Lexing)

Fungsi `tokenize()` memindai karakter HTML dari kiri ke kanan dan menghasilkan token-token berikut:

- Token open: tag pembuka beserta atributnya (misal `<div class="box" id="main">`).
- Token close: tag penutup (misal `</div>`).
- Token text: konten teks di antara tag.
- Token comment: komentar HTML (`<!-- ... -->`), diabaikan saat build tree.
- Token doctype: deklarasi `<!DOCTYPE html>`, diabaikan.

Tokenizer menangani void elements (br, img, input, dll.) yang tidak memerlukan tag penutup, serta raw text elements (script, style) yang kontennya tidak diparsing sebagai HTML.

3.2.2. Pembangunan Pohon (Tree Building)

Fungsi `parseTree()` mengonsumsi token-token yang dihasilkan tokenizer dan membangun array flat `treeNode[]`. Menggunakan variabel `curRoot` (ID node yang sedang aktif sebagai parent):

- Token open: buat node baru, jadikan anak dari `curRoot`, lalu pindah `curRoot` ke node baru (kecuali void/self-closing).
- Token close: naiki pohon (cari ancestor dengan `tagName` yang sesuai) dan pindah `curRoot` ke sana.
- Token text: tambahkan konten teks ke `curRoot`.

3.3. Pencocokan Selektor CSS (`matchesSelector`)

Setelah pohon dibangun, fungsi `matchesSelector(nodes, nodeId, selector)` dievaluasi untuk setiap node yang dikunjungi selama BFS/DFS. Implementasi di `algo.ts` mendukung selektor kompleks melalui fungsi rekursif `checkSelectorMatch()`:

3.3.1. Selektor Sederhana (`matchesSimpleSelector`)

Mencocokkan satu bagian selektor tanpa combinator:

- Selektor `'*'`: cocok dengan semua node.
- Dimulai `'.'`: cocok jika node memiliki class yang sesuai.
- Dimulai `'#'`: cocok jika node memiliki id yang sesuai.
- Lainnya: ekstrak tag, class, dan id dari selektor, lalu cocokkan masing-masing.

3.3.2. Selektor Kompleks dengan Combinator

Fungsi `checkSelectorMatch()` memecah selektor menjadi bagian-bagian berdasarkan spasi dan simbol combinator (`>`, `+`, `~`), lalu secara rekursif mengevaluasi dari kanan ke kiri:

- Descendant (spasi): cek apakah ada ancestor yang cocok dengan sisa selektor.
- Child (`>`): cek apakah parent langsung cocok dengan sisa selektor.

- Adjacent (+): cek apakah sibling sebelumnya cocok dengan sisa selektor.
- General sibling (~): cek apakah ada sibling sebelumnya yang cocok.

3.4. Aplikasi BFS pada DOM

Ketika pengguna memilih BFS dan memasukkan selektor CSS, fungsi `searchBFS()` dipanggil dari root (ID=0). Algoritma mengunjungi node level demi level.

Implikasi praktis BFS pada DOM: jika pengguna mencari `<nav>`, BFS akan menemukan elemen navigasi yang terletak di level atas (shallow) sebelum menemukan elemen `<nav>` yang tersarang jauh di dalam konten. Ini berguna untuk menemukan elemen layout utama.

3.5. Aplikasi DFS pada DOM

Ketika pengguna memilih DFS, fungsi `searchDFS()` dipanggil. Algoritma langsung menyelami cabang pertama (kiri) sedalam mungkin sebelum berpindah ke cabang berikutnya. Urutan kunjungan DFS pada pohon DOM bersifat pre-order: kunjungi node, lalu anak-anaknya dari kiri ke kanan.

Implikasi praktis DFS pada DOM: DFS sesuai dengan perilaku `querySelector()` standar browser. Jika selektor hanya cocok dengan satu elemen, DFS dan browser akan memberikan hasil yang identik. DFS lebih efisien dalam hal memori untuk pohon DOM yang lebar (banyak anak).

3.6. Perbandingan Hasil Traversal

Meskipun keduanya mengunjungi semua node dan menemukan semua elemen yang cocok, urutan kunjungan dan urutan hasil berbeda:

- BFS: hasil diurutkan berdasarkan kedalaman (elemen paling dangkal muncul lebih awal).
- DFS: hasil diurutkan berdasarkan urutan pre-order (elemen di cabang kiri muncul lebih awal).

Untuk selektor seperti 'span' pada halaman Wikipedia (test.html, 653 span), kedua algoritma menemukan semua 653 span, tetapi dengan urutan kemunculan berbeda dalam traversalLog.

Bab 4

Implementasi dan Pengujian

4.1. Implementasi

4.1.1 Backend

algo.ts

```
import { treeNode } from "../tree";

export function matchesSimpleSelector(node: treeNode, selector: string):
boolean {
  if (!node) return false;
  if (selector === "") return true;

  if (selector.startsWith(".")) {
    const classes = node.className ?
node.className.split(/\s+/).filter(Boolean) : [];
    return selector.split(".").filter(Boolean).every(c =>
classes.includes(c));
  }

  if (selector.startsWith("#")) {
    return node.idName === selector.slice(1);
  }

  const tagEndIdx = selector.search(/[.#]/);
  const tag = (tagEndIdx === -1 ? selector : selector.slice(0,
tagEndIdx)).toLowerCase();

  if (tag && tag !== "*" && node.tagName.toLowerCase() !== tag) return false;

  const classMatches = [...selector.matchAll(/\.([^.#]+)/g)];
  if (classMatches.length > 0) {
    const classes = node.className ?
node.className.split(/\s+/).filter(Boolean) : [];
    for (const m of classMatches) {
      if (!classes.includes(m[1])) return false;
    }
  }

  const idMatch = selector.match(/#([^.#]+)/);
  if (idMatch && node.idName !== idMatch[1]) return false;

  return true;
}

export function matchesSelector(nodes: treeNode[], nodeId: number, selector:
string): boolean {
```

```

    const normalized = selector.trim().replace(/\s*([>~])\s*/g, " $1 ");
    const parts = normalized.split(/\s+/).filter(Boolean);
    if (parts.length === 1) return matchesSimpleSelector(nodes[nodeId],
selector.trim());
    return checkSelectorMatch(nodes, nodeId, parts);
}

function checkSelectorMatch(nodes: treeNode[], nodeId: number, parts:
string[]): boolean {
    if (parts.length === 0) return true;

    const node = nodes[nodeId];
    if (!matchesSimpleSelector(node, parts[parts.length - 1])) return false;
    if (parts.length === 1) return true;

    const op = parts[parts.length - 2];
    const rest = parts.slice(0, parts.length - 2);

    if (op === ">") {
        if (node.parent === null) return false;
        return checkSelectorMatch(nodes, node.parent, rest);
    }

    if (op === "+") {
        if (node.parent === null) return false;
        const siblings = nodes[node.parent].children;
        const idx = siblings.indexOf(nodeId);
        if (idx <= 0) return false;
        return checkSelectorMatch(nodes, siblings[idx - 1], rest);
    }

    if (op === "~") {
        if (node.parent === null) return false;
        const siblings = nodes[node.parent].children;
        const idx = siblings.indexOf(nodeId);
        for (let j = 0; j < idx; j++) {
            if (checkSelectorMatch(nodes, siblings[j], rest)) return true;
        }
        return false;
    }

    // descendant combinator
    const ancestor = parts.slice(0, parts.length - 1);
    let parentId = node.parent;
    while (parentId !== null) {
        if (checkSelectorMatch(nodes, parentId, ancestor)) return true;
        parentId = nodes[parentId].parent;
    }
    return false;
}

export function searchBFS(nodes: treeNode[], startId: number, selector:
string, topN = Infinity) {
    const log: number[] = [];
    const results: number[] = [];
    const queue: number[] = [startId];

```

```

const start = performance.now();

while (queue.length > 0) {
  const id = queue.shift()!;
  const node = nodes[id];
  if (!node) continue;

  log.push(id);
  if (matchesSelector(nodes, id, selector)) {
    results.push(id);
    if (results.length >= topN) break;
  }

  for (const child of node.children) queue.push(child);
}

return { results, traversalLog: log, visited: log.length, time:
performance.now() - start };
}

export function searchDFS(nodes: treeNode[], startId: number, selector:
string, topN = Infinity) {
  const log: number[] = [];
  const results: number[] = [];
  const stack: number[] = [startId];
  const start = performance.now();

  while (stack.length > 0) {
    const id = stack.pop()!;
    const node = nodes[id];
    if (!node) continue;

    log.push(id);
    if (matchesSelector(nodes, id, selector)) {
      results.push(id);
      if (results.length >= topN) break;
    }

    for (let i = node.children.length - 1; i >= 0; i--) {
      stack.push(node.children[i]);
    }
  }

  return { results, traversalLog: log, visited: log.length, time:
performance.now() - start };
}

```

lca.ts

```

import { treeNode } from "../tree";

const LOG_BITS = 17;

export interface LCATable {
  depth: number[]; // depth[v]
  up: number[][]; // up[k][v] =

```

```

    n:      number;
}

export function buildLCATable(nodes: treeNode[]): LCATable {
    const n = nodes.length;
    const depth = new Array<number>(n).fill(0);
    const up: number[][] = Array.from({ length: LOG_BITS }, () => new
Array<number>(n).fill(-1));

    // BFS dari root
    const visited = new Uint8Array(n);
    const queue: number[] = [0];
    visited[0] = 1;
    up[0][0] = 0;

    while (queue.length > 0) {
        const u = queue.shift()!;
        for (const c of nodes[u].children) {
            if (c < n && !visited[c]) {
                visited[c] = 1;
                depth[c] = depth[u] + 1;
                up[0][c] = u;
                queue.push(c);
            }
        }
    }

    // bangun sparse table
    for (let k = 1; k < LOG_BITS; k++) {
        for (let v = 0; v < n; v++) {
            const p = up[k - 1][v];
            up[k][v] = (p === -1) ? -1 : up[k - 1][p];
        }
    }

    return { depth, up, n };
}

export function queryLCA(
    table: LCATable,
    uIn: number,
    vIn: number
): {
    lca:      number;
    pathU:    number[]; // node-node dari u naik ke LCA (inklusif kedua ujung)
    pathV:    number[]; // node-node dari v naik ke LCA (inklusif kedua ujung)
    depthU:   number;
    depthV:   number;
    depthLCA: number;
} {
    const { depth, up } = table;
    let u = uIn, v = vIn;

    const pathU: number[] = [];
    const pathV: number[] = [];

    // pastikan u lebih dalam
    const swapped = depth[u] < depth[v];
    if (swapped) { [u, v] = [v, u]; }

    // leveling

```

```

let diff = depth[u] - depth[v];
pathU.push(u);
for (let k = 0; k < LOG_BITS; k++) {
  if ((diff >> k) & 1) {
    u = up[k][u];
    if (u !== -1) pathU.push(u);
  }
}

if (u === v) {
  const lcaId = v;
  if (swapped) {
    return {
      lca: lcaId,
      pathU: pathV,          // swap balik ke input order
      pathV: pathU,
      depthU: depth[vIn],
      depthV: depth[uIn],
      depthLCA: depth[lcaId],
    };
  }
  return {
    lca: lcaId,
    pathU,
    pathV,
    depthU: depth[uIn],
    depthV: depth[vIn],
    depthLCA: depth[lcaId],
  };
}

// binary lifting
pathV.push(v);
for (let k = LOG_BITS - 1; k >= 0; k--) {
  if (up[k][u] !== up[k][v]) {
    u = up[k][u];
    v = up[k][v];
    if (u !== -1) pathU.push(u);
    if (v !== -1) pathV.push(v);
  }
}

const lcaId = up[0][u];
pathU.push(lcaId);

if (swapped) {
  return {
    lca: lcaId,
    pathU: pathV,
    pathV: pathU,
    depthU: depth[vIn],
    depthV: depth[uIn],
    depthLCA: depth[lcaId],
  };
}
return {
  lca: lcaId,
  pathU,
  pathV,
  depthU: depth[uIn],
  depthV: depth[vIn],
};

```

```

        depthLCA: depth[lcaId],
    };
}

export function pathFromRoot(table: LCATable, nodeId: number): number[] {
    const { up } = table;
    const path: number[] = [];
    let cur = nodeId;
    while (true) {
        path.push(cur);
        const p = up[0][cur];
        if (p === cur) break;    // root
        cur = p;
    }
    return path.reverse();
}

```

mt.ts

```

import { Worker, isMainThread, parentPort, workerData } from "worker_threads";
import * as path from "path";
import { treeNode } from "../tree";
import { searchBFS, searchDFS } from "../algo";

const N_WORKERS          = 4;
const SPLIT_DEPTH        = 3;
const MIN_NODES_FOR_MT = 500;    // di bawah ini single-thread selalu lebih
cepat

interface PoolWorker {
    worker: Worker;
    busy: boolean;
    resolve: ((v: WorkerResult) => void) | null;
    reject: ((e: Error) => void) | null;
}

interface WorkerTask {
    nodes: treeNode[];
    startId: number;
    selector: string;
    algo: "BFS" | "DFS";
    topN: number;
}

interface WorkerResult {
    log: number[];
    results: number[];
}

let pool: PoolWorker[] | null = null;
const taskQueue: Array<{ task: WorkerTask; resolve: (v: WorkerResult) => void;
reject: (e: Error) => void }> = [];

function workerScriptPath(): string {
    const base = path.join(__dirname, "worker");
    try { require.resolve(base + ".js"); return base + ".js"; } catch { return
base + ".ts"; }
}

```

```

function makePoolWorker(): PoolWorker {
  const wp = workerScriptPath();
  const execArgv = wp.endsWith(".ts") ? ["--require", "ts-node/register"] :
  [];

  // pool mode: worker standby
  const worker = new Worker(wp, {
    workerData: { poolMode: true },
    execArgv,
  });

  const pw: PoolWorker = { worker, busy: false, resolve: null, reject: null };

  worker.on("message", (result: WorkerResult) => {
    // selesai, ambil task berikutnya
    const res = pw.resolve;
    pw.resolve = null;
    pw.reject = null;
    pw.busy = false;
    res?.(result);
    drainQueue();
  });

  worker.on("error", (err) => {
    pw.busy = false;
    pw.reject?.(err);
    pw.resolve = null;
    pw.reject = null;
  });

  return pw;
}

function getPool(): PoolWorker[] {
  if (!pool) {
    pool = Array.from({ length: N_WORKERS }, makePoolWorker);
  }
  return pool;
}

function drainQueue() {
  if (taskQueue.length === 0) return;
  const freeWorker = getPool().find(pw => !pw.busy);
  if (!freeWorker) return;

  const { task, resolve, reject } = taskQueue.shift()!;
  freeWorker.busy = true;
  freeWorker.resolve = resolve;
  freeWorker.reject = reject;
  freeWorker.worker.postMessage(task);
}

function runWorker(task: WorkerTask): Promise<WorkerResult> {
  return new Promise((resolve, reject) => {
    const freeWorker = getPool().find(pw => !pw.busy);
  });
}

```

```

    if (freeWorker) {
      freeWorker.busy = true;
      freeWorker.resolve = resolve;
      freeWorker.reject = reject;
      freeWorker.worker.postMessage(task);
    } else {
      taskQueue.push({ task, resolve, reject });
    }
  });
}

function collectFrontier(
  nodes: treeNode[],
  splitDepth: number
): { frontierLog: number[]; frontier: number[] } {
  const frontierLog: number[] = [];
  const frontier: number[] = [];
  const queue: Array<{ id: number; depth: number }> = [{ id: 0, depth: 0 }];

  while (queue.length > 0) {
    const { id, depth } = queue.shift()!;
    const node = nodes[id];
    if (!node) continue;
    frontierLog.push(id);

    if (depth >= splitDepth || node.children.length === 0) {
      frontier.push(id);
    } else {
      for (const c of node.children) {
        queue.push({ id: c, depth: depth + 1 });
      }
    }
  }
  return { frontierLog, frontier };
}

export async function searchMT(
  nodes: treeNode[],
  selector: string,
  algo: "BFS" | "DFS",
  topN: number
): Promise<{ results: number[]; traversalLog: number[]; visited: number; time: number; threads: number }> {
  const start = performance.now();

  // pohon kecil, pakai single-thread
  if (nodes.length < MIN_NODES_FOR_MT) {
    const r = algo === "BFS"
      ? searchBFS(nodes, 0, selector, topN)
      : searchDFS(nodes, 0, selector, topN);
    return { ...r, threads: 1 };
  }

  // Kumpulkan frontier
  const { frontierLog, frontier } = collectFrontier(nodes, SPLIT_DEPTH);

```



```

// bagi frontier ke N_WORKERS bucket (round-robin)
const buckets: number[][] = Array.from({ length: N_WORKERS }, () => []);
frontier.forEach((id, i) => buckets[i % N_WORKERS].push(id));

// tiap bucket dijalankan satu worker
const workerPromises = buckets
  .filter(b => b.length > 0)
  .map(bucket =>
    (async (): Promise<WorkerResult> => {
      const log: number[] = [];
      const results: number[] = [];
      for (const startId of bucket) {
        const r = await runWorker({ nodes, startId, selector, algo, topN });
        log.push(...r.log);
        results.push(...r.results);
        if (results.length >= topN) break;
      }
      return { log, results };
    })()
  );

const workerResults = await Promise.all(workerPromises);

const traversalLog: number[] = [...frontierLog];
const allResults: number[] = [];
for (const wr of workerResults) {
  traversalLog.push(...wr.log);
  allResults.push(...wr.results);
}

return {
  results: allResults.slice(0, topN === Infinity ? undefined : topN),
  traversalLog,
  visited: traversalLog.length,
  time: performance.now() - start,
  threads: workerPromises.length,
};
}

```

lca.ts

```

import { treeNode, makeNode } from "./tree";

const voidElements = new Set([
  "area", "base", "br", "col", "embed",
  "hr", "img", "input", "link", "meta",
  "param", "source", "track", "wbr"
]);

const rawTextElements = new Set(["script", "style"]);

export interface Token {
  type: "open" | "close" | "text" | "comment" | "doctype";
  name?: string;
  attrs?: Map<string, string>;
  content?: string;
  selfClose?: boolean;
}

```

```

function parseAttrs(attrStr: string): Map<string, string> {
  const attrs = new Map<string, string>();
  const re =
/((^\\s"'>/=]+)\\s*(?:\\s*("([\\^"]*)"'|'([\\^']*)'|([\\s"'>/=]*)))?/g;
  let m: RegExpExecArray | null;
  while ((m = re.exec(attrStr)) !== null) {
    attrs.set(m[1].toLowerCase(), m[2] ?? m[3] ?? m[4] ?? "");
  }
  return attrs;
}

function tokenize(html: string): Token[] {
  const tokens: Token[] = [];
  let i = 0;
  const len = html.length;

  while (i < len) {
    if (html[i] !== "<") {
      const start = i;
      while (i < len && html[i] !== "<") i++;
      const text = html.slice(start, i).trim();
      if (text) tokens.push({ type: "text", content: text });
      continue;
    }

    i++;

    if (i + 2 < len && html[i] === "!" && html[i + 1] === "-" && html[i + 2]
=== "-") {
      const end = html.indexOf("-->", i + 3);
      if (end === -1) { i = len; break; }
      tokens.push({ type: "comment", content: html.slice(i + 3, end) });
      i = end + 3;
      continue;
    }

    if (i < len && html[i] === "!") {
      const end = html.indexOf(">", i);
      if (end === -1) { i = len; break; }
      tokens.push({ type: "doctype" });
      i = end + 1;
      continue;
    }

    let tagContent = "";
    let inQuote = "";
    while (i < len) {
      const ch = html[i];
      if (inQuote) {
        tagContent += ch;
        if (ch === inQuote) inQuote = "";
      } else if (ch === '"' || ch === "'") {
        inQuote = ch;
        tagContent += ch;
      } else if (ch === ">") {
        i++;
        break;
      } else {
        tagContent += ch;
      }
    }
  }
}

```

```

        i++;
    }

    tagContent = tagContent.trim();
    if (!tagContent) continue;

    if (tagContent[0] === "/" ) {
        tokens.push({ type: "close", name:
tagContent.slice(1).trim().split(/\s+/)[0].toLowerCase() });
        continue;
    }

    const selfClose = tagContent.endsWith("/");
    const cleanTag = selfClose ? tagContent.slice(0, -1).trim() : tagContent;
    const firstSpace = cleanTag.search(/\s/);
    const name = (firstSpace === -1 ? cleanTag : cleanTag.slice(0,
firstSpace)).toLowerCase();
    const attrStr = firstSpace === -1 ? "" : cleanTag.slice(firstSpace + 1);
    const isVoid = voidElements.has(name);

    tokens.push({ type: "open", name, attrs: parseAttrs(attrStr), selfClose:
selfClose || isVoid });

    if (rawTextElements.has(name) && !selfClose && !isVoid) {
        const closeTag = `</${name}`;
        const closeIdx = html.toLowerCase().indexOf(closeTag, i);
        if (closeIdx !== -1) {
            const raw = html.slice(i, closeIdx).trim();
            if (raw) tokens.push({ type: "text", content: raw });
            i = html.indexOf(">", closeIdx) + 1;
            tokens.push({ type: "close", name });
        } else {
            i = len;
        }
    }
}

return tokens;
}

export function parseHTML(html: string): Token[] {
    return tokenize(html);
}

export function parseTree(tokens: Token[]): treeNode[] {
    const nodes: treeNode[] = [];
    nodes.push(makeNode(0, "#root", "", "", "", null));

    let nodeCount = 0;
    let curRoot = 0;

    for (const token of tokens) {
        if (token.type === "doctype" || token.type === "comment") continue;

        if (token.type === "text") {
            if (token.content?.trim()) {
                const sep = nodes[curRoot].content ? " " : "";
                nodes[curRoot].content += sep + token.content.trim();
            }
            continue;
        }
    }
}

```

```
if (token.type === "close") {
  let t = curRoot;
  while (t !== 0 && nodes[t].tagName !== token.name) {
    t = nodes[t].parent ?? 0;
  }
  if (nodes[t].tagName === token.name) {
    curRoot = nodes[t].parent ?? 0;
  }
  continue;
}

if (token.type === "open") {
  nodeCount++;
  nodes.push(makeNode(
    nodeCount,
    token.name!,
    "",
    token.attrs?.get("class") ?? "",
    token.attrs?.get("id") ?? "",
    curRoot
  ));
  nodes[curRoot].children.push(nodeCount);
  if (!token.selfClose) curRoot = nodeCount;
}
}

return nodes;
}
```

scraper.ts

```

import axios from "axios";
import https from "https";

const httpsAgent = new https.Agent({ rejectUnauthorized: false });

export async function scrapeHTML(url: string): Promise<string | null> {
    let finalUrl = url.trim();
    if (!finalUrl.startsWith("http://") && !finalUrl.startsWith("https://")) {
        finalUrl = "https://" + finalUrl;
    }

    console.log(`[Scraper] Mengambil data dari: ${finalUrl}`);

    try {
        const response = await axios.get(finalUrl, {
            headers: {
                "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36",
                "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
                "Accept-Language": "en-US,en;q=0.9"
            },
            timeout: 15000,
            responseType: "text",
            maxRedirects: 10,
            httpsAgent
        });

        const html = typeof response.data === "string" ? response.data :
String(response.data);
        return html;
    } catch (error: any) {
        const msg = error.response
            ? `HTTP ${error.response.status}: ${error.response.statusText}`
            : error.message || "Gagal fetch";
        console.error("[Scraper] Gagal mendapatkan HTML:", msg);
        return null;
    }
}

```

tree.ts

```

export interface treeNode {
    id: number;
    tagName: string;
    content: string;
    className: string;
    idName: string;
    parent: number | null;
    children: number[];
}

export function makeNode(
    id: number,
    tagName: string,
    content: string,
    className: string,
    idName: string,
    parent: number | null
): treeNode {
    return { id, tagName, content, className, idName, parent, children: [] };
}

```

```

export function getMaxDepth(nodes: treeNode[], id: number): number {
  const node = nodes[id];
  if (!node || node.children.length === 0) return 1;
  let max = 0;
  for (const cid of node.children) {
    const d = getMaxDepth(nodes, cid);
    if (d > max) max = d;
  }
  return 1 + max;
}

```

worker.ts

```

import { workerData, parentPort } from "worker_threads";
import { treeNode } from "../tree";
import { matchesSelector } from "../algo";

interface Task {
  nodes:    treeNode[];
  startId:  number;
  selector: string;
  algo:     "BFS" | "DFS";
  topN:     number;
}

function runTask(task: Task): { log: number[]; results: number[] } {
  const { nodes, startId, selector, algo, topN } = task;
  const log:    number[] = [];
  const results: number[] = [];

  if (algo === "BFS") {
    const queue: number[] = [startId];
    while (queue.length > 0 && results.length < topN) {
      const id  = queue.shift()!;
      const node = nodes[id];
      if (!node) continue;
      log.push(id);
      if (matchesSelector(nodes, id, selector)) results.push(id);
      for (const c of node.children) queue.push(c);
    }
  } else {
    const stack: number[] = [startId];
    while (stack.length > 0 && results.length < topN) {
      const id  = stack.pop()!;
      const node = nodes[id];
      if (!node) continue;
      log.push(id);
      if (matchesSelector(nodes, id, selector)) results.push(id);
      for (let i = node.children.length - 1; i >= 0; i--) {
        stack.push(node.children[i]);
      }
    }
  }

  return { log, results };
}

if (workerData?.poolMode) {
  parentPort!.on("message", (task: Task) => {
    const result = runTask(task);
    parentPort!.postMessage(result);
  });
} else {

```

```

const result = runTask(workerData as Task);
parentPort!.postMessage(result);
}

```

4.1.2. Frontend

index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>TUAK DOM Visualizer</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link
href="https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@400;500;600;
700&family=JetBrains+Mono:wght@400;600&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="app-container">

    <header class="app-header">
      <div class="header-left">
        <div class="header-logo brand-logos">
          
          
        </div>

        <div class="header-text">
          <h1>TUAK DOM VISUALIZER</h1>
          <p class="header-sub">Created by TUAK, Juan (13524032),
Ariel (13524085), Manu (13524102)</p>
        </div>

        <div class="header-badges">
          <span class="badge badge-bfs">BFS</span>
          <span class="badge badge-dfs">DFS</span>
        </div>
      </header>

      <div class="panel step-panel" id="step1-panel">
        <div class="panel-title">
          <span class="panel-icon">

            <svg width="16" height="16" viewBox="0 0 16 16"
fill="none">
              <path d="M6.5 9.5a4 4 0 0 0 5.657 0l1.414-1.414a4 4 0
0 0-5.657-5.657L6.5 3.843" stroke="#3d7a35" stroke-width="1.5"
stroke-linecap="round"/>
              <path d="M9.5 6.5a4 4 0 0 0-5.657 0L2.43 7.914a4 4 0 0
0 5.657 5.657L9.5 12.157" stroke="#3d7a35" stroke-width="1.5"

```

```

stroke-linecap="round"/>
    </svg>
  </span>
  1. Input HTML
</div>
<div class="input-row">
  <div class="input-col">
    <label>
      <svg width="13" height="13" viewBox="0 0 13 13"
fill="none" style="vertical-align:middle;margin-right:4px">
        <circle cx="6.5" cy="6.5" r="5.5" stroke="#3d7a35"
stroke-width="1.2"/>
        <path d="M4 6.5h5M6.5 4v5" stroke="#3d7a35"
stroke-width="1.2" stroke-linecap="round"/>
      </svg>
      URL (Link)
    </label>
    <input
type="text" id="url-input"
placeholder="https://example.com">
  </div>
  <div class="input-col">
    <label>
      <svg width="13" height="13" viewBox="0 0 13 13"
fill="none" style="vertical-align:middle;margin-right:4px">
        <rect x="1.5" y="1.5" width="10" height="10"
rx="2" stroke="#3d7a35" stroke-width="1.2"/>
        <path d="M3.5 4.5h6M3.5 6.5h4M3.5 8.5h5"
stroke="#3d7a35" stroke-width="1.2" stroke-linecap="round"/>
      </svg>
      Raw HTML <span
class="label-optional">(Optional)</span>
    </label>
    <textarea id="html-input" rows="3" placeholder="Atau paste
kode HTML di sini..."></textarea>
  </div>
  <button id="parse-btn" type="button" class="action-btn">
    <svg width="15" height="15" viewBox="0 0 15 15"
fill="none" style="vertical-align:middle;margin-right:5px">
      <path d="M2 7.5h11M9 3.5l4 4-4 4"
stroke="currentColor" stroke-width="1.8" stroke-linecap="round"
stroke-linejoin="round"/>
    </svg>
    Parse
  </button>
</div>
</div>

<div class="panel step-panel disabled" id="step2-panel">
  <div class="panel-title">
    <span class="panel-icon">
      <svg width="16" height="16" viewBox="0 0 16 16"
fill="none">
        <path d="M2 4h12M2 8h12M2 12h12" stroke="#3d7a35"
stroke-width="1.5" stroke-linecap="round"/>
        <circle cx="5" cy="4" r="1.8" fill="#fff"
stroke="#3d7a35" stroke-width="1.2"/>
        <circle cx="11" cy="8" r="1.8" fill="#fff"
stroke="#3d7a35" stroke-width="1.2"/>

```



```

        <circle cx="7" cy="12" r="1.8" fill="#fff"
stroke="#3d7a35" stroke-width="1.2"/>
    </svg>
</span>
    2. Traversal Options
</div>
<div class="control-row">
    <div class="control-item">
        <label>
            <svg width="13" height="13" viewBox="0 0 13 13"
fill="none" style="vertical-align:middle;margin-right:4px">
                <path d="M2 4h2.5a2 2 0 0 1 2 2v1a2 2 0 0 2
2H11" stroke="#3d7a35" stroke-width="1.2" stroke-linecap="round"/>
                <circle cx="11" cy="9" r="1.5" fill="#3d7a35"/>
                <circle cx="2" cy="4" r="1.5" fill="#3d7a35"/>
            </svg>
            CSS Selector
        </label>
        <input type="text" id="selector-input" placeholder="div,
.class, #id, div.foo">
    </div>
    <div class="control-item">
        <label>
            <svg width="13" height="13" viewBox="0 0 13 13"
fill="none" style="vertical-align:middle;margin-right:4px">
                <path d="M6.5 1v11M1 6.5h11" stroke="#3d7a35"
stroke-width="1.2" stroke-linecap="round"/>
                <circle cx="6.5" cy="6.5" r="5" stroke="#3d7a35"
stroke-width="1.2"/>
            </svg>
            Algoritma
        </label>
        <select id="algo-select" title="Pilih algoritma
traversal">
            <option value="BFS">BFS – Breadth First</option>
            <option value="DFS">DFS – Depth First</option>
        </select>
    </div>
    <div class="control-item">
        <label>
            <svg width="13" height="13" viewBox="0 0 13 13"
fill="none" style="vertical-align:middle;margin-right:4px">
                <rect x="1.5" y="3.5" width="10" height="7"
rx="1.5" stroke="#3d7a35" stroke-width="1.2"/>
                <path d="M4.5 3.5V2.5a2 2 0 0 1 4 0v1"
stroke="#3d7a35" stroke-width="1.2" stroke-linecap="round"/>
            </svg>
            Hasil Ditampilkan
        </label>
        <div class="limit-controls">
            <label class="radio-label">
                <input type="radio" name="limit-type" value="all"
checked> Semua
            </label>
            <label class="radio-label">
                <input type="radio" name="limit-type"
value="topn"> Top
            <input type="number" id="limit-n" value="10"
min="1" class="limit-n-input" disabled>
        </label>
    </div>

```

```

        </div>
        <div class="control-item">
            <label>
                <svg width="13" height="13" viewBox="0 0 13 13"
fill="none" style="vertical-align:middle;margin-right:4px">
                    <circle cx="6.5" cy="6.5" r="5" stroke="#3d7a35"
stroke-width="1.2"/>
                    <path d="M6.5 3.5v3l2 1.5" stroke="#3d7a35"
stroke-width="1.2" stroke-linecap="round" stroke-linejoin="round"/>
                </svg>
                Kecepatan Animasi: <span id="speed-label"
style="color:#f9a825;margin-left:4px;font-family:'JetBrains
Mono',monospace">20ms</span>
            </label>
            <input type="range" id="speed-select" min="1" max="100"
value="20" class="speed-slider" title="Geser untuk mengatur kecepatan
animasi">
            <div class="speed-hint"><span>Cepat
(1ms)</span><span>Lambat (100ms)</span></div>
        </div>
        <div class="btn-group">
            <label class="mt-toggle" title="Aktifkan multithreading
(Worker Threads) untuk BFS/DFS">
                <input type="checkbox" id="mt-checkbox">
                <span class="mt-toggle-label">
                    <svg width="13" height="13" viewBox="0 0 13 13"
fill="none" style="vertical-align:middle;margin-right:3px">
                        <path d="M2 3h9M2 6.5h9M2 10h9"
stroke="currentColor" stroke-width="1.3" stroke-linecap="round"/>
                        <circle cx="5" cy="3" r="1.3"
fill="currentColor"/>
                        <circle cx="8" cy="6.5" r="1.3"
fill="currentColor"/>
                        <circle cx="4" cy="10" r="1.3"
fill="currentColor"/>
                    </svg>
                    Multithreading
                </span>
            </label>
            <button id="traverse-btn" type="button" class="action-btn
btn-orange" disabled>
                <svg width="15" height="15" viewBox="0 0 15 15"
fill="none" style="vertical-align:middle;margin-right:5px">
                    <path d="M3 7.5C3 4.46 5.46 2 8.5 2a5.5 5.5 0 0 1
0 11C5.46 13 3 10.54 3 7.5z" stroke="currentColor" stroke-width="1.6"/>
                    <path d="M6.5 5.5l3 2-3 2V5.5z"
fill="currentColor"/>
                </svg>
                Traverse
            </button>
            <button id="lca-btn" type="button" class="action-btn
btn-lca" disabled>
                <svg width="15" height="15" viewBox="0 0 15 15"
fill="none" style="vertical-align:middle;margin-right:5px">
                    <circle cx="7.5" cy="2.5" r="2"
stroke="currentColor" stroke-width="1.4"/>
                    <circle cx="3" cy="11" r="2"
stroke="currentColor" stroke-width="1.4"/>
                    <circle cx="12" cy="11" r="2"
stroke="currentColor" stroke-width="1.4"/>
                    <path d="M7.5 4.5v2M5.5 8.5L3 9M9.5 8.5L12 9"

```

```

stroke="currentColor" stroke-width="1.3" stroke-linecap="round"/>
    </svg>
    Traverse LCA
  </button>
</div>
</div>
</div>

  <div class="lca-status-bar" id="lca-status-bar" style="display:none">
    <svg width="16" height="16" viewBox="0 0 16 16"
fill="none"><circle cx="8" cy="4" r="2.5" stroke="#3d7a35"
stroke-width="1.5"/><circle cx="3" cy="13" r="2" stroke="#3d7a35"
stroke-width="1.3"/><circle cx="13" cy="13" r="2" stroke="#3d7a35"
stroke-width="1.3"/><path d="M8 6.5v3M5.5 11L3 11M10.5 11L13 11"
stroke="#3d7a35" stroke-width="1.3" stroke-linecap="round"/></svg>
    <span id="lca-status-text">Mode LCA aktif – Klik <b>2 node</b>
pada pohon DOM, lalu tekan <b>Traverse LCA</b></span>
    <div class="lca-node-chips">
      <div class="lca-chip" id="lca-chip-1">Node 1:
<span></span></div>
      <div class="lca-chip" id="lca-chip-2">Node 2:
<span></span></div>
    </div>
    <button class="lca-cancel-btn" id="lca-cancel-btn">X
Batal</button>
  </div>

  <div class="workspace">
    <div class="tree-section panel">
      <div class="tree-header">
        <div class="panel-title" style="margin-bottom:0">
          <span class="panel-icon">

            <svg width="16" height="16" viewBox="0 0 16 16"
fill="none">
              <circle cx="8" cy="3" r="2" fill="#3d7a35"/>
              <circle cx="4" cy="10" r="2" fill="#8fca7a"/>
              <circle cx="12" cy="10" r="2" fill="#8fca7a"/>
              <line x1="8" y1="5" x2="4" y2="8"
stroke="#3d7a35" stroke-width="1.4"/>
              <line x1="8" y1="5" x2="12" y2="8"
stroke="#3d7a35" stroke-width="1.4"/>
              <line x1="8" y1="5" x2="8" y2="13"
stroke="#3d7a35" stroke-width="1.4" stroke-dasharray="2 1.5"/>
            </svg>
          </span>
          DOM Tree
        </div>
        <div class="zoom-controls">
          <button id="zoom-out-btn" class="zoom-btn" title="Zoom
Out"></button>
          <span id="zoom-label">100%</span>
          <button id="zoom-in-btn" class="zoom-btn" title="Zoom
In"></button>
          <button id="zoom-reset-btn" class="zoom-btn"
title="Reset Zoom">&#8635;</button>
        </div>
      </div>
    </div>
  </div>
</div>
<div id="tree-viewport">

```

```

        <div id="tree-canvas"></div>
    </div>

    <div class="empty-state" id="empty-state">
        <svg width="60" height="60" viewBox="0 0 60 60"
fill="none">
            <circle cx="30" cy="12" r="7" fill="#e8f5e0"
stroke="#8fca7a" stroke-width="2"/>
            <circle cx="15" cy="36" r="7" fill="#e8f5e0"
stroke="#8fca7a" stroke-width="2"/>
            <circle cx="45" cy="36" r="7" fill="#e8f5e0"
stroke="#8fca7a" stroke-width="2"/>
            <line x1="30" y1="19" x2="15" y2="29" stroke="#c5e8b0"
stroke-width="2"/>
            <line x1="30" y1="19" x2="45" y2="29" stroke="#c5e8b0"
stroke-width="2"/>
            <text x="27" y="16" font-size="8" fill="#3d7a35"
font-family="monospace">&lt;/&gt;</text>
        </svg>
        <p>Parse HTML dulu untuk melihat pohon DOM</p>
    </div>

    <div class="log-section panel">
        <div class="panel-title">
            <span class="panel-icon">

                <svg width="16" height="16" viewBox="0 0 16 16"
fill="none">
                    <rect x="2" y="2" width="12" height="12" rx="2"
stroke="#3d7a35" stroke-width="1.4"/>
                    <path d="M5 5.5h6M5 8h4M5 10.5h5" stroke="#3d7a35"
stroke-width="1.3" stroke-linecap="round"/>
                </svg>
            </span>
            Traversal Log
        </div>
        <div class="stats-bar" id="stats-info">
            <span class="stat-item">
                <svg width="12" height="12" viewBox="0 0 12 12"
fill="none" style="vertical-align:middle"><circle cx="6" cy="6" r="4.5"
stroke="#3d7a35" stroke-width="1.2"/><path d="M6 3v3l2 1" stroke="#3d7a35"
stroke-width="1.1" stroke-linecap="round"/></svg>
                Nodes: 0
            </span>
            <span class="stat-divider">|</span>
            <span class="stat-item">
                <svg width="12" height="12" viewBox="0 0 12 12"
fill="none" style="vertical-align:middle"><path d="M6 1v10M2 5l4-4 4 4"
stroke="#3d7a35" stroke-width="1.2" stroke-linecap="round"
stroke-linejoin="round"/></svg>
                Depth: 0
            </span>
        </div>
        <ul id="log-list">
            <li class="log-placeholder">
                <svg width="20" height="20" viewBox="0 0 20 20"
fill="none" style="display:block;margin:0 auto 6px"><circle cx="10" cy="10"
r="8" stroke="#c5e8b0" stroke-width="1.5"/><path d="M7 10h6M10 7v6"
stroke="#8fca7a" stroke-width="1.5" stroke-linecap="round"/></svg>
                Log penelusuran akan muncul di sini...
            </li>
        </ul>
    </div>

```

```

        </li>
      </ul>
    </div>
  </div>

  </div>
  <script src="script.js"></script>
</body>
</html>

```

script.js

```

const API_BASE = window.location.hostname
  ? `${window.location.protocol}//${window.location.hostname}:3000`
  : 'http://localhost:3000';

const urlInput      = document.getElementById('url-input');
const htmlInput     = document.getElementById('html-input');
const parseBtn      = document.getElementById('parse-btn');
const step2Panel    = document.getElementById('step2-panel');
const selectorInput = document.getElementById('selector-input');
const algoSelect    = document.getElementById('algo-select');
const traverseBtn   = document.getElementById('traverse-btn');
const lcaBtn        = document.getElementById('lca-btn');
const speedSelect   = document.getElementById('speed-select');
const speedLabel    = document.getElementById('speed-label');
const limitNInput   = document.getElementById('limit-n');
const treeContainer = document.getElementById('tree-canvas');
const treeViewport  = document.getElementById('tree-viewport');
const logList       = document.getElementById('log-list');
const statsInfo     = document.getElementById('stats-info');
const zoomLabel     = document.getElementById('zoom-label');
const lcaStatusBar  = document.getElementById('lca-status-bar');
const lcaChip1      = document.getElementById('lca-chip-1');
const lcaChip2      = document.getElementById('lca-chip-2');
const lcaCancelBtn  = document.getElementById('lca-cancel-btn');

let savedHtmlSource = "";
let currentNodesData = [];
let nodeElements    = {};
let zoomLevel       = 1.0;

let lcaMode      = false;
let lcaSelected  = [];

const LOG_BITS = 16;
let lcaDepth   = [];
let lcaParent  = [];

const ZOOM_MIN      = 0.2;
const ZOOM_MAX      = 3.0;
const ZOOM_STEP_BTN = 0.2;
const ZOOM_STEP_WHEEL = 0.08;
const MAX_LOG_ENTRIES = 500;

speedSelect.addEventListener('input', () => {
  speedLabel.textContent = speedSelect.value + 'ms';
});

function applyZoom() {
  treeContainer.style.transform = `scale(${zoomLevel})`;
  const baseW = parseInt(treeContainer.style.width) || 3000;
  const baseH = parseInt(treeContainer.style.height) || 3000;

```

```

    treeContainer.style.marginRight = (baseW * zoomLevel - baseW) + 'px';
    treeContainer.style.marginBottom = (baseH * zoomLevel - baseH) + 'px';
    zoomLabel.innerText = Math.round(zoomLevel * 100) + '%';
}

function zoomAt(newZoom, pivotX, pivotY) {
    newZoom = Math.min(ZOOM_MAX, Math.max(ZOOM_MIN,
parseFloat(newZoom.toFixed(2))));
    if (newZoom === zoomLevel) return;
    const canvasX = (treeViewport.scrollLeft + pivotX) / zoomLevel;
    const canvasY = (treeViewport.scrollTop + pivotY) / zoomLevel;
    zoomLevel = newZoom;
    applyZoom();
    treeViewport.scrollLeft = canvasX * newZoom - pivotX;
    treeViewport.scrollTop = canvasY * newZoom - pivotY;
}

document.getElementById('zoom-in-btn').addEventListener('click', () =>
    zoomAt(zoomLevel + ZOOM_STEP_BTN, treeViewport.clientWidth / 2,
treeViewport.clientHeight / 2));
document.getElementById('zoom-out-btn').addEventListener('click', () =>
    zoomAt(zoomLevel - ZOOM_STEP_BTN, treeViewport.clientWidth / 2,
treeViewport.clientHeight / 2));
document.getElementById('zoom-reset-btn').addEventListener('click', () => {
zoomLevel = 1.0; applyZoom(); });

treeViewport.addEventListener('wheel', (e) => {
    e.preventDefault();
    const rect = treeViewport.getBoundingClientRect();
    zoomAt(zoomLevel + (e.deltaY > 0 ? -ZOOM_STEP_WHEEL : ZOOM_STEP_WHEEL),
        e.clientX - rect.left, e.clientY - rect.top);
}, { passive: false });

let isPanning = false;
let panStart = { x: 0, y: 0, sl: 0, st: 0 };

treeViewport.addEventListener('mousedown', (e) => {
    if (e.button !== 0) return;
    if (lcaMode && e.target.classList.contains('node-container')) return;
    isPanning = true;
    panStart = { x: e.clientX, y: e.clientY, sl: treeViewport.scrollLeft, st:
treeViewport.scrollTop };
    treeViewport.style.cursor = 'grabbing';
    e.preventDefault();
});

window.addEventListener('mousemove', (e) => {
    if (!isPanning) return;
    treeViewport.scrollLeft = panStart.sl - (e.clientX - panStart.x);
    treeViewport.scrollTop = panStart.st - (e.clientY - panStart.y);
});

window.addEventListener('mouseup', () => {
    if (!isPanning) return;
    isPanning = false;
    treeViewport.style.cursor = 'grab';
});

document.querySelectorAll('input[name="limit-type"]').forEach(radio => {
    radio.addEventListener('change', () => {
        limitNInput.disabled = radio.value !== 'topn';
    });
});
});

```

```

parseBtn.addEventListener('click', async () => {
  const url      = urlInput.value.trim();
  const rawHTML = htmlInput.value.trim();

  if (!url && !rawHTML) {
    alert("Masukkan URL atau paste HTML terlebih dahulu.");
    return;
  }

  parseBtn.innerText = "Parsing...";
  parseBtn.disabled  = true;
  treeContainer.innerHTML = '';
  zoomLevel = 1.0;
  applyZoom();
  logList.innerHTML = '<li class="log-placeholder">Menyiapkan DOM
Tree...</li>';

  const es = document.getElementById('empty-state');
  if (es) es.style.display = 'none';

  try {
    const response = await fetch(`${API_BASE}/parse`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ url, html: rawHTML })
    });

    const result = await response.json();
    if (result.success) {
      const data = result.data;
      savedHtmlSource = data.htmlSource || rawHTML;
      currentNodesData = data.nodes;

      if (url && data.htmlSource) htmlInput.value = data.htmlSource;

      buildLCAFrontend(data.nodes);

      visualizeTree(data.nodes);

      statsInfo.innerHTML =
        `<span class="stat-item">Nodes: ${data.nodes.length}</span>
        <span class="stat-divider">|</span>
        <span class="stat-item">Max Depth: ${data.maxDepth}</span>`;

      logList.innerHTML = '<li class="log-placeholder">Parsing sukses!
Pilih algoritma lalu klik Traverse.</li>';
      step2Panel.classList.remove('disabled');
      traverseBtn.disabled = false;
      lcaBtn.disabled      = false;
    } else {
      alert("Gagal parse: " + result.error);
    }
  } catch(e) {
    alert(`Koneksi ke backend gagal.\nURL: ${API_BASE}\n\n` + e);
  } finally {
    parseBtn.innerText = "Parse";
    parseBtn.disabled  = false;
  }
});

```

```

traverseBtn.addEventListener('click', async () => {
  const selector = selectorInput.value.trim();
  const algo      = algoSelect.value;

  if (!selector) {
    alert("Masukkan CSS Selector (contoh: div, .class, #id, a.nav)");
    return;
  }

  const limitType =
document.querySelector('input[name="limit-type"]:checked').value;
  const limit     = limitType === 'topn' ? Math.max(1,
parseInt(limitNInput.value) || 10) : 0;

  const mtCheckbox = document.getElementById('mt-checkbox');
  const useMT      = mtCheckbox ? mtCheckbox.checked : false;

  traverseBtn.innerText = "Running...";
  traverseBtn.disabled  = true;
  logList.innerHTML     = '';

  resetAllNodes();

  try {
    const response = await fetch(`${API_BASE}/search`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        html: savedHtmlSource,
        selector,
        algo,
        limit,
        mt: useMT,
      })
    });

    const result = await response.json();
    if (result.success) {
      const data = result.data;

      await animateAndLog(data.traversalLog, data.results);

      const threads = data.threads || 1;
      const summary = document.createElement('li');
      summary.innerHTML =
        `<b>Selesai!</b> Ditemukan <b>${data.results.length}</b>
kecocokan ` +
        `dari <b>${data.visited}</b> node dikunjungi. ` +
        `Waktu: <b>${data.time.toFixed(2)}ms</b>` +
        `(threads > 1 ? ` | Threads: <b>${threads}</b>` : '');
      summary.style.cssText =
'margin-top:10px;text-align:center;padding:8px;background:#e8f5e0;border-radius:6px;';
      logList.appendChild(summary);
      logList.scrollTop = logList.scrollHeight;
    } else {
      alert("Error traversal: " + result.error);
    }
  } catch(e) {
    alert("Terjadi kesalahan saat traversal.\n\n" + e);
  }
}

```



```

    } finally {
        traverseBtn.innerText = "Traverse";
        traverseBtn.disabled = false;
    }
});

lcaBtn.addEventListener('click', () => {
    if (!lcaMode) {
        lcaMode = true;
        lcaSelected = [];
        lcaStatusBar.style.display = 'flex';
        resetAllNodes();
        updateLCACHips();
        logList.innerHTML = '<li class="log-placeholder"> Mode LCA - Klik 2
node pada pohon lalu tekan <b>Traverse LCA</b>.</li>';
    } else {
        if (lcaSelected.length < 2) {
            alert("Pilih 2 node pada pohon terlebih dahulu!");
            return;
        }
        runLCA();
    }
});

lcaCancelBtn.addEventListener('click', () => {
    exitLCAMode();
    resetAllNodes();
});

function exitLCAMode() {
    lcaMode = false;
    lcaSelected = [];
    lcaStatusBar.style.display = 'none';
}

function attachNodeClickHandler(id, el) {
    el.addEventListener('click', (e) => {
        if (!lcaMode) return;
        e.stopPropagation();

        const idx = lcaSelected.indexOf(id);
        if (idx !== -1) {
            lcaSelected.splice(idx, 1);
            el.classList.remove('node-lca-selected');
        } else {
            if (lcaSelected.length >= 2) {
                const old = lcaSelected.shift();
                if (nodeElements[old])
nodeElements[old].classList.remove('node-lca-selected');
            }
            lcaSelected.push(id);
            el.classList.add('node-lca-selected');
        }
        updateLCACHips();
    });
}

function updateLCACHips() {
    const n0 = lcaSelected[0];
    const n1 = lcaSelected[1];
    const tag0 = n0 !== undefined ? `&lt;${currentNodesData[n0]?.tagName}&gt;

```

```

#{$n0}` : '-';
    const tag1 = n1 !== undefined ? `&lt;${currentNodesData[n1]?.tagName}&gt;` : '-';
#{$n1}` : '-';
    lcaChip1.innerHTML = `Node 1: <span>${tag0}</span>`;
    lcaChip2.innerHTML = `Node 2: <span>${tag1}</span>`;
    lcaChip1.classList.toggle('filled', n0 !== undefined);
    lcaChip2.classList.toggle('filled', n1 !== undefined);
}

function resetAllNodes() {
    Object.values(nodeElements).forEach(el => {
        el.className = 'node-container';
    });
}

function buildLCAFrontend(nodes) {
    const n = nodes.length;
    lcaDepth = new Array(n).fill(0);
    lcaParent = Array.from({ length: LOG_BITS }, () => new Array(n).fill(-1));

    const visited = new Array(n).fill(false);
    const queue = [0];
    visited[0] = true;
    lcaParent[0][0] = 0;

    while (queue.length > 0) {
        const u = queue.shift();
        for (const c of (nodes[u].children || [])) {
            if (c < n && !visited[c]) {
                visited[c] = true;
                lcaDepth[c] = lcaDepth[u] + 1;
                lcaParent[0][c] = u;
                queue.push(c);
            }
        }
    }

    for (let k = 1; k < LOG_BITS; k++) {
        for (let v = 0; v < n; v++) {
            const p = lcaParent[k-1][v];
            lcaParent[k][v] = p === -1 ? -1 : lcaParent[k-1][p];
        }
    }
}

async function runLCA() {
    const [u, v] = lcaSelected;
    const delay = parseInt(speedSelect.value);

    logList.innerHTML = '';
    const loadingItem = document.createElement('li');
    loadingItem.innerHTML = `<b>Menghitung LCA</b> untuk Node <b>#{$u}</b>
&amp; Node <b>#{$v}</b>...`;
    loadingItem.style.cssText =
'padding:6px;background:#e8f5e0;border-radius:6px;margin-bottom:4px;text-align:center;';
    logList.appendChild(loadingItem);

    try {
        const response = await fetch(`${API_BASE}/lca`, {
            method: 'POST',

```

```

        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            html: savedHtmlSource,
            nodeU: u,
            nodeV: v,
        })
    });

    const result = await response.json();

    if (!result.success) {
        alert("LCA gagal: " + result.error);
        exitLCAMode();
        return;
    }

    const { lca, lcaTag, lcaDepth: depthLCA, depthU, depthV,
            pathU, pathV, pathURoot, pathVRoot, time } = result.data;

    resetAllNodes();
    logList.innerHTML = '';

    const header = document.createElement('li');
    header.innerHTML =
        `<b>LCA Binary Lifting</b> - Node <b>#${u}</b> ` +
        `(depth:${depthU}) &amp; Node <b>#${v}</b> (depth:${depthV})`;
    header.style.cssText =
'padding:6px;background:#e8f5e0;border-radius:6px;margin-bottom:4px;text-align:center;';
    logList.appendChild(header);

    for (const id of pathU) {
        if (id === lca) continue; // LCA ditampilkan terakhir
        if (nodeElements[id]) {
            nodeElements[id].classList.add('node-lca-path');
            addLogItem(`↑ Path U: Node #${id}
<lt;${currentNodesData[id]?.tagName}>>`, '#5a8a3a');
            if (delay > 0) await sleep(delay);
        }
    }

    for (const id of pathV) {
        if (id === lca) continue;
        if (nodeElements[id]) {
            nodeElements[id].classList.add('node-lca-path');
            addLogItem(`↑ Path V: Node #${id}
<lt;${currentNodesData[id]?.tagName}>>`, '#3a7a8a');
            if (delay > 0) await sleep(delay);
        }
    }

    if (nodeElements[u]) {
        nodeElements[u].classList.remove('node-lca-path');
        nodeElements[u].classList.add('node-lca-selected');
    }
    if (nodeElements[v]) {
        nodeElements[v].classList.remove('node-lca-path');
        nodeElements[v].classList.add('node-lca-selected');
    }

    if (nodeElements[lca]) {

```

```

        nodeElements[lca].classList.remove('node-lca-path',
'node-lca-selected');
        nodeElements[lca].classList.add('node-lca-result');
    }

    const breadcrumbU = pathURoot.map(id =>
`<span
class="bc-item">#${id}&lt;${currentNodesData[id]?.tagName}&gt;</span>`
    ).join(' > ');
    const breadcrumbV = pathVRoot.map(id =>
`<span
class="bc-item">#${id}&lt;${currentNodesData[id]?.tagName}&gt;</span>`
    ).join(' > ');

    const breadcrumbLi = document.createElement('li');
    breadcrumbLi.innerHTML =
`<div style="font-size:11px;margin-top:6px;color:#666">` +
`<b>Root → U:</b> ${breadcrumbU}<br>` +
`<b>Root → V:</b> ${breadcrumbV}</div>`;
    logList.appendChild(breadcrumbLi);

    const resultLi = document.createElement('li');
    resultLi.innerHTML =
`<b>LCA ditemukan:</b> Node <b>#${lca}</b> ` +
`&lt;${lcaTag}&gt; (Depth: ${depthLCA}) | ` +
`Waktu: <b>${time.toFixed(3)}ms</b>`;
    resultLi.style.cssText =
`margin-top:8px;padding:8px;background:#a9dfbf;border-radius:6px;`
+
`text-align:center;font-weight:600;`;
    logList.appendChild(resultLi);
    logList.scrollTop = logList.scrollHeight;

    } catch(e) {
        alert(`Koneksi ke backend gagal saat menghitung LCA.\nURL:
${API_BASE}\n\n` + e);
    }

    exitLCAMode();
}

function addLogItem(html, color = '#333') {
    const li = document.createElement('li');
    li.innerHTML = html;
    li.style.color = color;
    logList.appendChild(li);
    logList.scrollTop = logList.scrollHeight;
}

function sleep(ms) {
    return new Promise(r => setTimeout(r, ms));
}

function visualizeTree(nodes) {
    const NODE_W = 110;
    const LEVEL_H = 90;
    nodeElements = {};

    const visitOrder = [];
    {
        const stk = [0];

```

```

        while (stk.length > 0) {
            const id = stk.pop();
            if (id == null || !nodes[id]) continue;
            visitOrder.push(id);
            const ch = nodes[id].children;
            for (let i = ch.length - 1; i >= 0; i--) stk.push(ch[i]);
        }
    }
    for (let i = visitOrder.length - 1; i >= 0; i--) {
        const node = nodes[visitOrder[i]];
        if (!node) continue;
        if (node.children.length === 0) {
            node.subtreeWidth = NODE_W + 20;
        } else {
            let total = 0;
            for (const cid of node.children) total +=
(nodes[cid]?.subtreeWidth || NODE_W + 20);
            node.subtreeWidth = Math.max(total, NODE_W + 20);
        }
    }

    const totalWidth = Math.max(3000, (nodes[0].subtreeWidth || 3000) + 200);
    const maxD = lcaDepth.length > 1 ? Math.max(...lcaDepth) : 20;
    const TOP_PAD = 120;
    const totalHeight = Math.max(3000, (maxD + 3) * (LEVEL_H + 10) + TOP_PAD);
    treeContainer.style.width = totalWidth + 'px';
    treeContainer.style.height = totalHeight + 'px';

    nodes[0].x = Math.max(1500, (nodes[0].subtreeWidth || 3000) / 2);
    nodes[0].y = TOP_PAD;

    const bfsQ = [0];
    while (bfsQ.length > 0) {
        const id = bfsQ.shift();
        const node = nodes[id];
        if (!node) continue;

        const isRoot = id === 0;

        if (!isRoot) {
            const div = document.createElement('div');
            div.className = 'node-container';
            div.style.left = Math.round(node.x - 50) + 'px';
            div.style.top = Math.round(node.y) + 'px';
            div.title = `<${node.tagName}> (ID: ${id}, Depth:
${lcaDepth[id] ?? '?'})`;
            div.innerText = `<${node.tagName}>`;
            treeContainer.appendChild(div);
            nodeElements[id] = div;
            attachNodeClickHandler(id, div);
        }

        const childY = isRoot ? node.y : node.y + LEVEL_H;
        let curX = node.x - (node.subtreeWidth || NODE_W + 20) / 2;
        for (const cid of node.children) {
            if (!nodes[cid]) continue;
            const w = nodes[cid].subtreeWidth || NODE_W + 20;
            nodes[cid].x = curX + w / 2;
            nodes[cid].y = childY;
            curX += w;
            bfsQ.push(cid);
        }
    }

```

```

    }
    drawLines();
}

function drawLines() {
    const svg = document.createElementNS("http://www.w3.org/2000/svg", "svg");
    svg.setAttribute("style",
"position:absolute;top:0;left:0;width:100%;height:100%;pointer-events:none;z-i
ndex:0;");
    treeContainer.prepend(svg);

    for (const node of currentNodesData) {
        if (!nodeElements[node.id] && node.id !== 0) continue;
        for (const cid of node.children) {
            if (!nodeElements[cid]) continue;
            const child = currentNodesData[cid];
            const midY = (node.y + 30 + child.y) / 2;
            const path =
document.createElementNS("http://www.w3.org/2000/svg", "path");
            path.setAttribute("d",
`M ${Math.round(node.x)} ${Math.round(node.y + 30)} L
${Math.round(node.x)} ${Math.round(midY)} L ${Math.round(child.x)}
${Math.round(midY)} L ${Math.round(child.x)} ${Math.round(child.y)}`);
            path.setAttribute("stroke", "#333");
            path.setAttribute("stroke-width", "1.5");
            path.setAttribute("fill", "none");
            svg.appendChild(path);
        }
    }
}

async function animateAndLog(logIds, targetIds) {
    const delay = parseInt(speedSelect.value);
    const targets = new Set(targetIds);
    const useAnim = delay > 0 && logIds.length <= 2000;

    let step = 1;
    for (const id of logIds) {
        if (id === 0) continue;
        const node = currentNodesData[id];
        if (!node) { step++; continue; }

        const isMatch = targets.has(id);
        const el = nodeElements[id];

        if (el) {
            el.classList.remove('node-visited', 'node-match');
            el.classList.add(isMatch ? 'node-match' : 'node-visited');
        }

        if (step <= MAX_LOG_ENTRIES) {
            const li = document.createElement('li');
            li.className = isMatch ? 'log-item-match' : 'log-item-visit';
            li.innerText = `[${step}] <${node.tagName}> (ID:${id})${isMatch ?
" [MATCH]" : ""}`;
            logList.appendChild(li);
            logList.scrollTop = logList.scrollHeight;
        } else if (step === MAX_LOG_ENTRIES + 1) {
            const li = document.createElement('li');

```

```

        li.className = 'log-placeholder';
        li.innerText = `... ${logIds.length - MAX_LOG_ENTRIES} langkah
lagi tidak ditampilkan`;
        logList.appendChild(li);
    }

    if (useAnim) await sleep(delay);
    step++;
}

if (!useAnim) await sleep(0);
}

```

style.css

```

/* === PALETTE ===
--green-dark   : #3d7a35
--green-light  : #8fca7a
--yellow       : #fff5a0
--orange       : #f9a825
*/

@import
url('https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@400;500;600;700&family=JetBrains+Mono:wght@400;600&display=swap');

* { box-sizing: border-box; margin: 0; padding: 0; }

body {
    font-family: 'Space Grotesk', Arial, sans-serif;
    background: #f0f7ee;
    background-image:
        radial-gradient(circle at 15% 10%, rgba(143,202,122,0.18) 0%,
transparent 45%),
        radial-gradient(circle at 85% 85%, rgba(249,168,37,0.10) 0%,
transparent 40%);
    color: #2a3d26;
    padding: 20px;
    min-height: 100vh;
    display: flex;
    justify-content: center;
}

.app-container {
    width: 100%;
    max-width: 1400px;
    display: flex;
    flex-direction: column;
    gap: 14px;
}

.app-header {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 16px;
    background: #3d7a35;
    border-radius: 14px;
    padding: 18px 24px;
    box-shadow: 0 4px 18px rgba(61,122,53,0.25);
}

```

```
.header-logo {
  flex-shrink: 0;
  filter: drop-shadow(0 2px 6px rgba(0,0,0,0.25));
}

.header-text { flex: 1; }

.header-text h1 {
  font-size: 22px;
  font-weight: 700;
  color: #fff5a0;
  letter-spacing: 0.06em;
  text-shadow: 0 1px 4px rgba(0,0,0,0.2);
  border: none;
  padding: 0;
}

.header-sub {
  font-size: 12px;
  color: rgba(255,255,255,0.72);
  margin-top: 3px;
  font-family: 'JetBrains Mono', monospace;
  letter-spacing: 0.02em;
}

.header-badges {
  display: flex;
  gap: 8px;
}

.badge {
  font-size: 11px;
  font-weight: 700;
  padding: 4px 10px;
  border-radius: 20px;
  letter-spacing: 0.06em;
  font-family: 'JetBrains Mono', monospace;
}

.badge-bfs {
  background: #fff5a0;
  color: #3d7a35;
}

.badge-dfs {
  background: #f9a825;
  color: #fff;
}

.panel {
  background: #fff;
  border: 1.5px solid #d4edca;
  border-radius: 12px;
  padding: 16px 20px;
  box-shadow: 0 2px 10px rgba(61,122,53,0.07);
  position: relative;
}

.panel-title {
```



```

    font-weight: 700;
    font-size: 12px;
    margin-bottom: 14px;
    color: #3d7a35;
    text-transform: uppercase;
    letter-spacing: 0.09em;
    display: flex;
    align-items: center;
    gap: 7px;
}

.panel-icon {
    display: flex;
    align-items: center;
    justify-content: center;
    background: #e8f5e0;
    border-radius: 6px;
    padding: 4px;
    flex-shrink: 0;
}

.input-row, .control-row {
    display: flex;
    gap: 12px;
    align-items: flex-end;
    flex-wrap: wrap;
}

.input-col { flex: 1; min-width: 180px; display: flex; flex-direction:
column; gap: 6px; }
.control-item { display: flex; flex-direction: column; gap: 6px; min-width:
200px; }

label {
    font-size: 12px;
    font-weight: 600;
    color: #4a6b44;
    display: flex;
    align-items: center;
    gap: 2px;
}

.label-optional {
    font-weight: 400;
    color: #9ab89a;
    font-size: 11px;
    margin-left: 3px;
}

input[type="text"],
input[type="number"],
textarea,
select {
    padding: 8px 10px;
    border: 1.5px solid #c5dfc0;
    border-radius: 7px;
    font-family: 'Space Grotesk', inherit;
    font-size: 13px;
    background: #fafef9;
    color: #2a3d26;
}

```

```

        transition: border-color 0.18s, box-shadow 0.18s;
        outline: none;
    }

    input[type="text"]:focus,
    input[type="number"]:focus,
    textarea:focus,
    select:focus {
        border-color: #3d7a35;
        box-shadow: 0 0 0 3px rgba(61,122,53,0.12);
    }

    textarea { resize: vertical; }

    /* Limit controls */
    .limit-controls {
        display: flex;
        align-items: center;
        gap: 10px;
        padding: 6px 10px;
        border: 1.5px solid #c5dfc0;
        border-radius: 7px;
        background: #f5fbf2;
        height: 37px;
    }

    .radio-label {
        display: flex;
        align-items: center;
        gap: 5px;
        font-weight: 500;
        color: #3a5438;
        font-size: 13px;
        cursor: pointer;
    }

    .radio-label input[type="radio"] {
        accent-color: #3d7a35;
        cursor: pointer;
    }

    .limit-n-input {
        width: 58px;
        padding: 2px 6px;
        height: 24px;
        font-size: 13px;
    }

    .limit-n-input:disabled { background: #eee; color: #aaa; }

    .action-btn {
        padding: 9px 20px;
        background: #3d7a35;
        color: #fff5a0;
        border: none;
        border-radius: 8px;
        font-family: 'Space Grotesk', inherit;
        font-weight: 700;
        font-size: 13px;
        cursor: pointer;
        height: 37px;
    }

```

```
white-space: nowrap;
align-self: flex-end;
display: flex;
align-items: center;
letter-spacing: 0.03em;
transition: background 0.18s, transform 0.1s, box-shadow 0.18s;
box-shadow: 0 3px 10px rgba(61,122,53,0.25);
}
.action-btn:hover {
  background: #2d5e27;
  transform: translateY(-1px);
  box-shadow: 0 5px 14px rgba(61,122,53,0.3);
}
.action-btn:active { transform: translateY(0); }
.action-btn:disabled {
  background: #b8ccb5;
  color: #fff;
  cursor: not-allowed;
  box-shadow: none;
  transform: none;
}

.btn-orange {
  background: #f9a825;
  color: #fff;
  box-shadow: 0 3px 10px rgba(249,168,37,0.3);
}
.btn-orange:hover {
  background: #e09820;
  box-shadow: 0 5px 14px rgba(249,168,37,0.35);
}
.btn-orange:disabled {
  background: #d4c090;
  color: #fff;
  box-shadow: none;
}

.disabled { opacity: 0.45; pointer-events: none; }

.workspace {
  display: flex;
  gap: 14px;
  flex: 1;
  min-height: 460px;
}

.tree-section {
  flex: 3;
  display: flex;
  flex-direction: column;
  overflow: hidden;
}

.tree-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 10px;
}
```

```

}

.zoom-controls {
  display: flex;
  align-items: center;
  gap: 5px;
}

.zoom-btn {
  width: 28px;
  height: 28px;
  border: 1.5px solid #c5dfc0;
  border-radius: 6px;
  background: #f5fbf2;
  color: #3d7a35;
  font-size: 16px;
  font-weight: 700;
  line-height: 1;
  cursor: pointer;
  display: flex;
  align-items: center;
  justify-content: center;
  transition: background 0.15s, border-color 0.15s;
}

.zoom-btn:hover {
  background: #e0f0da;
  border-color: #3d7a35;
}

#zoom-label {
  font-size: 12px;
  font-weight: 700;
  color: #3d7a35;
  min-width: 40px;
  text-align: center;
  font-family: 'JetBrains Mono', monospace;
}

#tree-viewport {
  flex: 1;
  position: relative;
  overflow: auto;
  background: #f8fdf6;
  background-image:
    linear-gradient(rgba(143,202,122,0.08) 1px, transparent 1px),
    linear-gradient(90deg, rgba(143,202,122,0.08) 1px, transparent 1px);
  background-size: 30px 30px;
  border: 1.5px solid #d4edca;
  border-radius: 8px;
  cursor: grab;
  user-select: none;
}

#tree-canvas {
  position: relative;
  width: 3000px;
  height: 3000px;
  transform-origin: top left;
  will-change: transform;
  backface-visibility: hidden;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

```

```

}

/* Empty state */
.empty-state {
  position: absolute;
  inset: 0;
  top: 50px;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 10px;
  pointer-events: none;
}

.empty-state p {
  font-size: 13px;
  color: #9ab89a;
  font-style: italic;
}

.log-section {
  flex: 1;
  display: flex;
  flex-direction: column;
  min-width: 220px;
}

.stats-bar {
  font-size: 12px;
  background: linear-gradient(135deg, #e8f5e0, #fff5a0 120%);
  border: 1px solid #c5dfc0;
  padding: 7px 12px;
  border-radius: 8px;
  margin-bottom: 10px;
  font-weight: 700;
  display: flex;
  align-items: center;
  gap: 8px;
  font-family: 'JetBrains Mono', monospace;
  color: #3d7a35;
}

.stat-item { display: flex; align-items: center; gap: 5px; }
.stat-divider { color: #c5dfc0; }

#log-list {
  flex: 1;
  overflow-y: auto;
  list-style: none;
  border: 1.5px solid #d4edca;
  border-radius: 8px;
  padding: 10px;
  background: #f8fdf6;
  font-size: 12px;
  font-family: 'JetBrains Mono', monospace;
}

#log-list li {
  padding: 4px 6px;
  border-bottom: 1px solid #eef7ea;
  border-radius: 3px;
  transition: background 0.1s;
}

```

```
}
#log-list li:last-child { border-bottom: none; }

.log-placeholder {
    color: #9ab89a;
    text-align: center;
    font-style: italic;
    padding: 20px 0 !important;
}
.log-item-visit {
    color: #c77c00;
    background: #ffffbea;
}
.log-item-match {
    color: #3d7a35;
    font-weight: 600;
    background: #e8f5e0 !important;
}

.btn-group {
    display: flex;
    flex-direction: column;
    gap: 7px;
    align-self: flex-end;
}

.btn-lca {
    background: #3d7a35;
    color: #fff5a0;
    box-shadow: 0 3px 10px rgba(61,122,53,0.25);
}
.btn-lca:hover {
    background: #2d5e27;
    box-shadow: 0 5px 14px rgba(61,122,53,0.3);
}
.btn-lca:disabled {
    background: #b8ccb5;
    color: #fff;
    box-shadow: none;
}

.speed-slider {
    -webkit-appearance: none;
    appearance: none;
    width: 100%;
    height: 6px;
    border-radius: 3px;
    background: linear-gradient(to right, #f9a825, #8fca7a);
    border: none !important;
    outline: none;
    padding: 0 !important;
    cursor: pointer;
    box-shadow: none !important;
}
.speed-slider::-webkit-slider-thumb {
    -webkit-appearance: none;
    width: 18px;
    height: 18px;
    border-radius: 50%;
```

```

    background: #3d7a35;
    border: 2.5px solid #fff;
    box-shadow: 0 1px 6px rgba(61,122,53,0.35);
    cursor: pointer;
    transition: transform 0.1s;
}
.speed-slider::-webkit-slider-thumb:hover { transform: scale(1.2); }
.speed-slider::-moz-range-thumb {
    width: 16px; height: 16px;
    border-radius: 50%;
    background: #3d7a35;
    border: 2px solid #fff;
    cursor: pointer;
}
.speed-hint {
    display: flex;
    justify-content: space-between;
    font-size: 10px;
    color: #9ab89a;
    font-family: 'JetBrains Mono', monospace;
    margin-top: 3px;
    width: 100%;
    min-width: 180px;
    gap: 8px;
}
.speed-hint span:last-child { text-align: right; }

.lca-status-bar {
    background: linear-gradient(135deg, #e8f5e0, #fff9e0);
    border: 1.5px solid #8fca7a;
    border-radius: 10px;
    padding: 10px 16px;
    display: flex;
    align-items: center;
    gap: 12px;
    flex-wrap: wrap;
    animation: slideDown 0.25s ease;
}
@keyframes slideDown {
    from { opacity: 0; transform: translateY(-8px); }
    to   { opacity: 1; transform: translateY(0); }
}
#lca-status-text {
    font-size: 13px;
    color: #3d7a35;
    flex: 1;
}
.lca-node-chips { display: flex; gap: 8px; }
.lca-chip {
    font-size: 11px;
    font-family: 'JetBrains Mono', monospace;
    padding: 4px 10px;
    border-radius: 20px;
    background: #fff;
    border: 1.5px solid #c5dfc0;
    color: #3d7a35;
    font-weight: 600;
    transition: border-color 0.2s, background 0.2s;
}
.lca-chip.filled {

```

```

        background: #fff5a0;
        border-color: #f9a825;
        color: #b07000;
    }
    .lca-cancel-btn {
        font-size: 11px;
        padding: 4px 10px;
        border: 1.5px solid #e0a0a0;
        border-radius: 6px;
        background: #fff5f5;
        color: #c0392b;
        cursor: pointer;
        font-weight: 600;
        transition: background 0.15s;
    }
    .lca-cancel-btn:hover { background: #ffe0e0; }

    .node-container {
        position: absolute;
        width: 100px;
        padding: 5px 6px;
        background: #fffde7;
        border: 2px solid #ffe082;
        border-radius: 6px;
        text-align: center;
        font-size: 11px;
        font-family: 'JetBrains Mono', monospace;
        font-weight: 600;
        z-index: 10;
        cursor: pointer;
        overflow: hidden;
        text-overflow: ellipsis;
        white-space: nowrap;
        color: #5a4a00;
        box-shadow: 0 1px 4px rgba(0,0,0,0.06);
        transition: border-color 0.15s, background 0.15s, box-shadow 0.15s,
transform 0.1s;
        -webkit-font-smoothing: antialiased;
        -moz-osx-font-smoothing: grayscale;
    }
    .node-container:hover {
        transform: scale(1.05);
        box-shadow: 0 3px 8px rgba(0,0,0,0.12);
        z-index: 20;
    }
}

/* Traversal states */
.node-visited {
    background: #fde8e8 !important;
    border-color: #e74c3c !important;
    color: #922b21 !important;
}
.node-match {
    background: #d5f5e3 !important;
    border-color: #27ae60 !important;
    color: #1a5e2a !important;
    box-shadow: 0 0 8px rgba(39,174,96,0.4) !important;
}

/* LCA states */

```



```

.node-lca-selected {
    background: #fff3cd !important;
    border-color: #f9a825 !important;
    border-width: 3px !important;
    color: #7d4e00 !important;
    box-shadow: 0 0 10px rgba(249,168,37,0.45) !important;
}
.node-lca-path {
    background: #d5f5e3 !important;
    border-color: #8fca7a !important;
    color: #2d7a40 !important;
}
.node-lca-result {
    background: #a9dfbf !important;
    border-color: #1e8449 !important;
    border-width: 3px !important;
    color: #0d4a24 !important;
    box-shadow: 0 0 14px rgba(30,132,73,0.5) !important;
    transform: scale(1.1) !important;
    z-index: 30 !important;
}

/* Scrollbar */
::-webkit-scrollbar { width: 6px; height: 6px; }
::-webkit-scrollbar-track { background: #f0f7ee; border-radius: 3px; }
::-webkit-scrollbar-thumb { background: #8fca7a; border-radius: 3px; }
::-webkit-scrollbar-thumb:hover { background: #3d7a35; }

.header-left {
    display: flex;
    align-items: center;
    gap: 14px;
}

.brand-logos {
    display: flex;
    align-items: center;
    gap: 8px;
}

.brand-logo {
    width: 42px;
    height: 42px;
    object-fit: contain;
    border-radius: 8px;
}

.hmif-logo {
    mix-blend-mode: multiply;
}

/* — Multithreading Toggle */

.mt-toggle {
    display: flex;
    align-items: center;
    gap: 6px;
    cursor: pointer;
    user-select: none;
}

.mt-toggle input[type="checkbox"] {

```

```

width: 15px;
height: 15px;
accent-color: #3d7a35;
cursor: pointer;
flex-shrink: 0;
}

.mt-toggle-label {
  font-size: 0.82rem;
  color: #555;
  font-weight: 500;
  white-space: nowrap;
}

.mt-toggle input:checked + .mt-toggle-label {
  color: #3d7a35;
  font-weight: 600;
}

/* — LCA Breadcrumb path items */

.bc-item {
  display: inline-block;
  background: #e8f5e0;
  border: 1px solid #b7dca0;
  border-radius: 4px;
  padding: 1px 5px;
  font-size: 10px;
  font-family: 'JetBrains Mono', monospace;
  color: #2d5a27;
  margin: 1px 0;
}

```

4.2. Pengujian

TC 1 : www.example.com

TUAK DOM VISUALIZER
 Created by TUAK, Juan (13524032), Ariel (13524085), Manu (13524102)

BFS DFS

1. INPUT HTML

URL (Link)

Raw HTML (Optional)

```
<!doctype html><html lang="en"><head><title>Example Domain</title><meta name="viewport" content="width=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}</style></head><body><div><h1><p><a>
```

Parse

2. TRAVERSAL OPTIONS

CSS Selector

Algoritma
 BFS — Breadth First

Hasil Ditampilkan
☒ Semua ☐ Top 10

Kecepatan Animasi: 63ms
 Cepat (1ms) — Lambat (100ms)

☐ Multithreading

Traverse

0% Traverse LCA

Mode LCA aktif — Klik 2 node pada pohon DOM, lalu tekan Traverse LCA

Node 1: — Node 2: — X Batal

DOM TREE

TRAVERSAL LOG

Nodes: 12 | Max Depth: 6

```
[1] <html> (ID:1)
[2] <head> (ID:2)
[3] <body> (ID:6)
[4] <title> (ID:3)
[5] <meta> (ID:4)
[6] <style> (ID:5)
[7] <div> (ID:7)
[8] <h1> (ID:8)
[9] <p> (ID:9) [MATCH]
[10] <p> (ID:10) [MATCH]
[11] <a> (ID:11)
```

Selesai! Ditemukan 2 kecocokan dari 12 node dikunjungi. Waktu: 0.27ms

TC2 : https://en.wikipedia.org/wiki/Wikipedia:Short_pages

2. TRAVERSAL OPTIONS

CSS Selector

div, class, #id, div.foo

Algoritma

BFS — Breadth First

Hasil Ditampilkan

Semua Top 10

Kecepatan Animasi: 20ms

Cepat (1ms) Lambat (100ms)

Multithreading

Traverse

Traverse LCA

DOM TREE

92%

<div>

<header>

<a>

<a>

<div>

<div>

<div>

<form>

<div>

<div>

<input>

TRAVERSAL LOG

Nodes: 430 | Max Depth: 21

Parsing sukses! Pilih algoritma lalu klik Traverse.

TC3 : <https://www.geeksforgeeks.org/dsa/dynamic-programming/>

1. INPUT HTML

URL (Link)

<https://www.geeksforgeeks.org/dsa/dynamic-programming/>

Raw HTML (Optional)

```
<!DOCTYPE html><html lang="en"><head><link rel="preconnect" href="https://fonts.googleapis.com/"><link rel="preconnect" href="https://fonts.gstatic.com" crossorigin="true"/><meta charset="UTF-8"/><meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0"></head><body></body></html>
```

2. TRAVERSAL OPTIONS

CSS Selector

a

Algoritma

DFS — Depth First

Hasil Ditampilkan

Semua Top 10

Kecepatan Animasi: 76ms

Cepat (1ms) Lambat (100ms)

Multithreading

☐

Traverse

0% Traverse LCA

Mode LCA aktif — Klik 2 node pada pohon DOM, lalu tekan Traverse LCA

Node 1: #575 Node 2: <i> #697

DOM TREE

28%

DOM Tree visualization showing a grid of nodes. The root node is at the top, and the tree structure is represented by a grid of nodes. The nodes are colored red and green, indicating different states or types of nodes.

TRAVERSAL LOG

Nodes: 875 Max Depth: 25

```
[1] <html> (ID:1)
[2] <head> (ID:2)
[3] <link> (ID:3)
[4] <link> (ID:4)
[5] <meta> (ID:5)
[6] <meta> (ID:6)
[7] <meta> (ID:7)
[8] <link> (ID:8)
[9] <meta> (ID:9)
[10] <meta> (ID:10)
[11] <meta> (ID:11)
[12] <meta> (ID:12)
[13] <meta> (ID:13)
[14] <meta> (ID:14)
[15] <meta> (ID:15)
[16] <meta> (ID:16)
[17] <meta> (ID:17)
[18] <meta> (ID:18)
[19] <meta> (ID:19)
[20] <meta> (ID:20)
[21] <meta> (ID:21)
[22] <meta> (ID:22)
[23] <meta> (ID:23)
[24] <meta> (ID:24)
[25] <meta> (ID:25)
[26] <meta> (ID:26)
[27] <meta> (ID:27)
[28] <meta> (ID:28)
[29] <script> (ID:29)
```

TC4 :

```
<!DOCTYPE html>
```

```
<html lang="en">
```

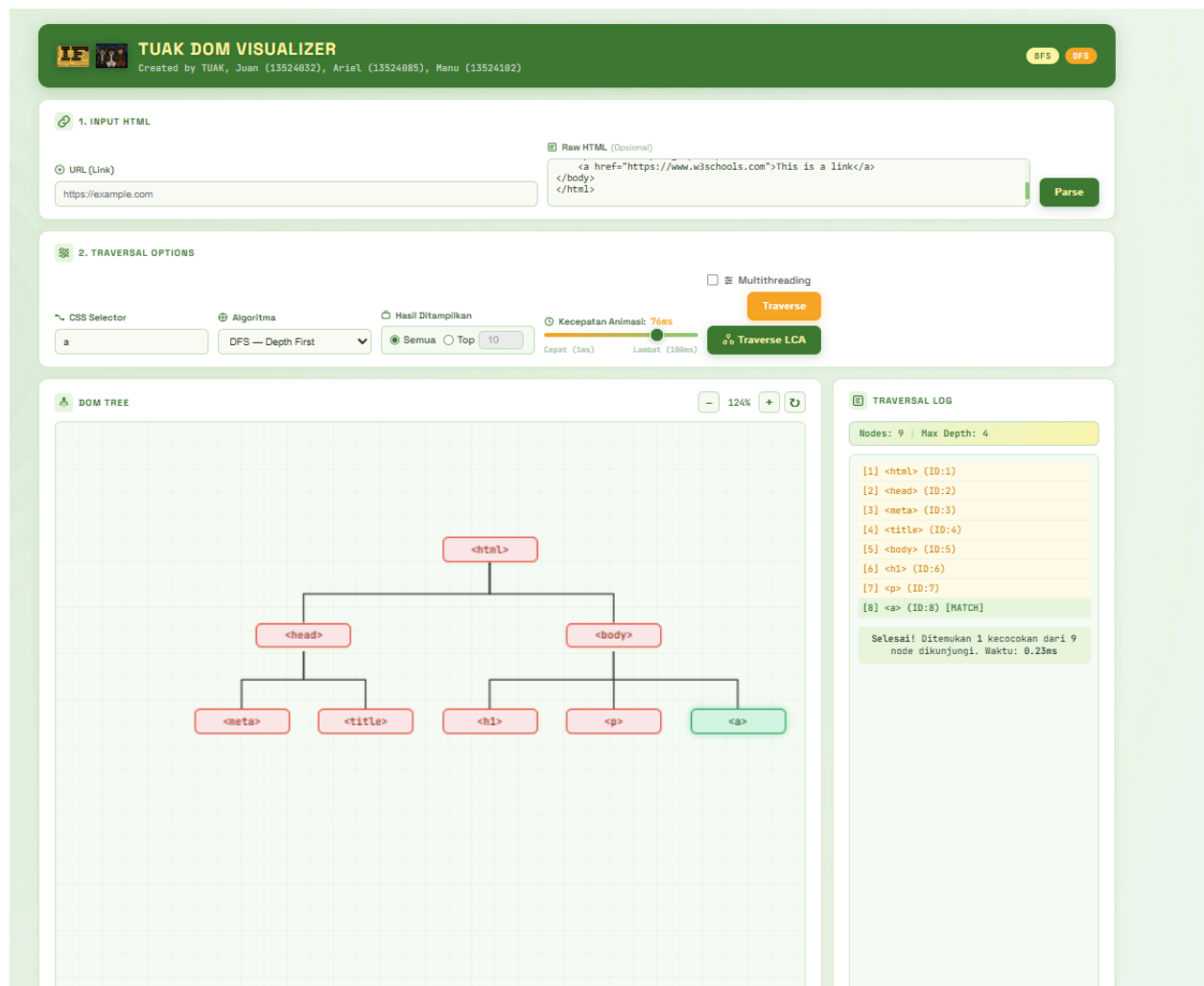
```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>Page Title</title>
```

```
</head>
```

```
<body>
```



Bab 5

Kesimpulan dan Saran

5.1. Kesimpulan

Berdasarkan implementasi dan pengujian yang telah dilakukan, dapat disimpulkan beberapa hal berikut:

- Algoritma BFS dan DFS keduanya efektif untuk penelusuran selektor CSS pada pohon DOM. Kedua algoritma mengunjungi seluruh N node dalam $O(N)$ waktu dan menghasilkan himpunan hasil yang identik. Perbedaan utamanya terletak pada urutan kunjungan: BFS menelusuri level demi level (urutan kedalaman menaik), sedangkan DFS menyelami cabang sepenuhnya (pre-order traversal).
- BFS lebih cocok digunakan ketika pengguna ingin menemukan elemen yang berada di level atas pohon DOM lebih cepat (misal: elemen navigasi, header). DFS lebih cocok digunakan ketika urutan kunjungan perlu konsisten dengan perilaku browser standar (querySelector) atau untuk pohon yang lebar dengan kedalaman terbatas.
- Implementasi LCA dengan Binary Lifting terbukti efisien: preprocessing $O(N \log N)$ hanya perlu dilakukan sekali per pohon, dan setiap query LCA diselesaikan dalam $O(\log N)$. Ini jauh lebih baik dari pendekatan naive $O(N)$ per query.
- Multithreading menggunakan Worker Threads tidak memberikan peningkatan performa untuk pohon DOM berukuran normal (ratusan hingga ribuan node), karena overhead komunikasi antar thread (serialisasi data, latency postMessage) melebihi waktu komputasi BFS/DFS itu sendiri. Multithreading baru menguntungkan untuk pohon yang sangat besar (>50.000 node).

5.2. Saran

Untuk UI masih terlalu sederhana dan untuk metode scrolling juga masih kurang efektif jika pohon DOM-nya sangat besar hingga ribuan node. Selebihnya sudah baik. Terimakasih juga kepada asisten yang sudah memperpanjang deadline pengerjaan.

Daftar Pustaka

- [1] GeeksforGeeks, “Breadth First Search or BFS for a Graph,” 2025. [Online]. Available: <https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/>
- [2] GeeksforGeeks, “Depth First Search or DFS for a Graph,” 2025. [Online]. Available: <https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/>
- [3] Mozilla Developer Network, “Document Object Model (DOM),” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
- [4] Mozilla Developer Network, “Document.querySelector(),” [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/Document/querySelector>
- [5] CP-Algorithms, “Lowest Common Ancestor - Binary Lifting,” [Online]. Available: https://cp-algorithms.com/graph/lca_binary_lifting.html
- [6] R. Munir, “Strategi Algoritma,” Institut Teknologi Bandung, 2023. [Online]. Available: <https://informatika.stei.itb.ac.id/~rinaldi.munir/Strategi-Algoritma/>
- [7] R. Munir, “Graf: BFS dan DFS,” Institut Teknologi Bandung, 2023. [Online]. Available: <https://informatika.stei.itb.ac.id/~rinaldi.munir/Struktur-Diskrit/>
- [8] W. Fiset, “Breadth First Search Algorithm (BFS),” YouTube, 2019. [Online]. Available: <https://www.youtube.com/watch?v=oDqjPvD54Ss>
- [9] W. Fiset, “Depth First Search Algorithm (DFS),” YouTube, 2019. [Online]. Available: <https://www.youtube.com/watch?v=7fujbpJ0LB4>
- [10] Fireship, “HTML Parsing in JavaScript,” YouTube, 2020. [Online]. Available: <https://www.youtube.com/watch?v=Vv5L9qU8p2A>
- [11] Web Dev Simplified, “DOM Explained,” YouTube, 2021. [Online]. Available: <https://www.youtube.com/watch?v=0ik6X4DJKCc>
- [12] Traversy Media, “Next.js Crash Course,” YouTube, 2022. [Online]. Available: <https://www.youtube.com/watch?v=mTz0GXj8NN0>
- [13] Programming with Mosh, “REST API Tutorial,” YouTube, 2021. [Online]. Available: <https://www.youtube.com/watch?v=SLwpqD8n3d0>

LAMPIRAN

No	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan		
2	Aplikasi berhasil dijalankan		
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil		
4	Aplikasi dapat melakukan scraping terhadap web pada input		
5	Aplikasi dapat menampilkan visualisasi pohon DOM		
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran		
7	Aplikasi dapat menandai jalur tempuh oleh algoritma		
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log		
9	[Bonus] Membuat video		
10	[Bonus] Deploy aplikasi		
11	[Bonus] Implementasi animasi pada penelusuran pohon		
12	[Bonus] Implementasi multithreading		
13	[Bonus] Implementasi LCA Binary Lifting		

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Manuel Thimoty Silalahi



Ariel Cornelius Sitorus



Juan Oloando Simanungkalit
[Tanda tangan mahasiswa]
[Nama mahasiswa]

Link Repository : https://github.com/manuellthimoty/Tubes2_Tuak